

Module «Data Science»

Week 01



mario.gellrich@zhaw.ch

Dr. Mario Gellrich

Table of contents

Content of this week

- Organizational information about the module «Data Science»
- Prerequisite Knowledge
- Minimal Software Requirements for this Module
- Self-Tests on Python & SQL
- What is Data Science?
- Recapitulation Management of Relational Data
- Recapitulation Management of Non-Relational (NoSQL) Data
- Management & Use of Real-time Data
 - Part I: Basics of Real-time Data & Event Streaming
 - Part II: Event Streaming with MQTT and Apache Kafka

Prerequisite Knowledge

- Statistics
- Python programming (procedural)
- Working with Visual Studio Code
- Working with Jupyter Notebooks
- Working with Git and GitHub
- File formats (e.g., XML, JSON), Data Management
- Database Queries with SQL

Prerequisite Knowledge: Helpful Resources

1. Python Tutorial

- [Python tutorial at W3Schools](#)

2. SQL & PostgreSQL Tutorial

- [SQL tutorial at W3Schools](#)
- [PostgreSQL tutorial at W3Schools](#)

3. NoSQL (MongoDB) Tutorial (part of this course)

- [MongoDB tutorial at W3Schools](#)

4. Programming Environments & Software

Visual Studio Code: [VS Code Basics](#)

Jupyter Notebooks: [Jupyter Notebooks Documentation](#)

Git & GitHub: [Git Introduction](#)

GitHub Codespaces: [GitHub Codespaces Overview](#)

Minimal Software and Services - Local and Cloud Setup

Anaconda with Python and Jupyter



Visual Studio Code



Visual Studio Code

Docker Desktop



Git



GitHub Account / GitHub Codespaces



ChatGPT, MS Copilot



ChatGPT

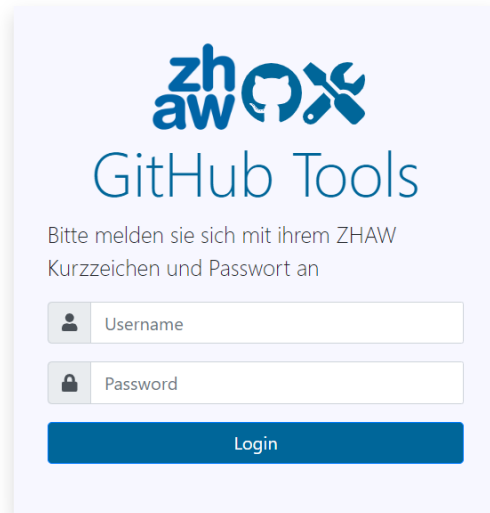


installations on local PC

Cloud environment
(GitHub Codespaces)

Minimal Software and Services - GitHub and Copilot Support

[ZHAW GitHub activation and support page](#)



The screenshot shows a login interface for 'GitHub Tools' at ZHAW. At the top is the ZHAW logo, which combines the letters 'zh' and 'aw' with a GitHub Octocat icon and a wrench icon. Below the logo, the text 'GitHub Tools' is displayed. A message in German asks the user to log in with their ZHAW short name and password. There are two input fields: one for 'Username' with a person icon and one for 'Password' with a lock icon. A blue 'Login' button is positioned below the password field.

Install the **Copilot** extension
in Visual Studio Code and
activate the [free] GitHub
Global campus licence.

[Additional information in case you need support ...](#)

Online Multiple Choice (Single Answer) Test on Python

Link to Python Quiz: [Open the Python self-test](#)



Report your % correct answers:



Online Multiple Choice (Single Answer) Test on SQL

Link to SQL Quiz: [Open the SQL self-test](#)



Report your % correct answers:

What Is Data Science? - Section Overview

What is Data Science?

Data Science – A Multidisciplinary Approach

The Three Primary Skills (Conway, 2010)

The Overlaps – and the Danger Zone

The Data Analytics Process Model

The Process Model in Context of this Module

Key Takeaways & Learning Objectives

What Is Data Science? - Definition and Relevance

Data Science is a multidisciplinary field that combines programming, mathematics & statistics, and domain expertise to extract knowledge and actionable insights from data.

Why now?

- Explosive growth of data: sensors, web, transactions, streams
- Cheap computing power and open-source tools (Python, SQL, ...)
- Organizations compete on data-driven decisions
- New data types: relational, NoSQL, and real-time data



Data

raw material: structured, semi-structured, streaming



Insights

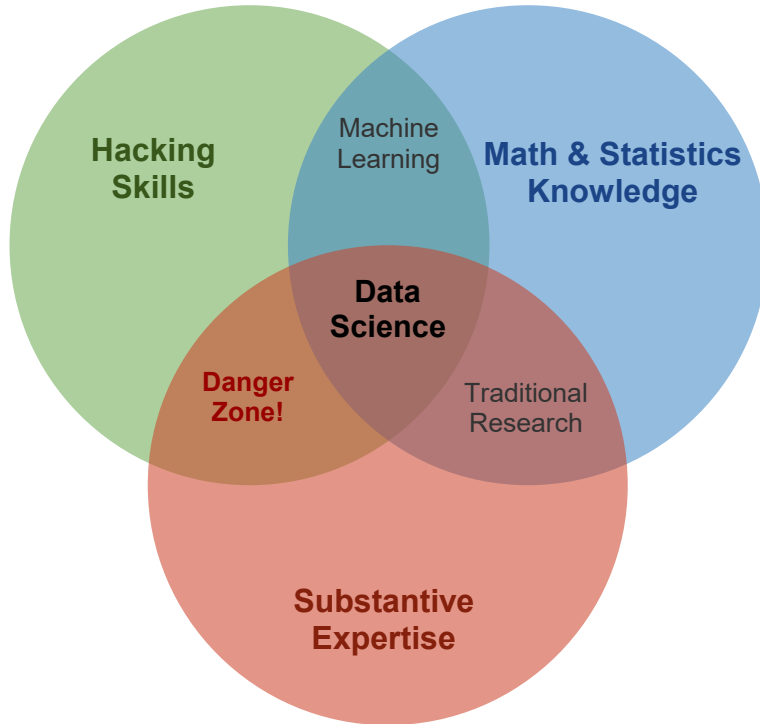
patterns, models, predictions



Decisions

actions that create value

Data Science – A Multidisciplinary Approach



How to read the diagram

- The three circles are the “primary colors” of data: hacking skills, math & statistics, substantive expertise.
- Each skill alone is valuable – combined with only one other, it is not yet data science.
- Data Science sits in the middle: all three competencies together.
- None of the skills is discipline-specific → inherently interdisciplinary.

Own illustration based on: Drew Conway (2010), The Data Science Venn Diagram

The Three Primary Skills (Conway, 2010)



Hacking Skills

- Data is acquired and traded electronically – you must “speak hacker”
- Manipulating (text) files, command line, APIs
- Vectorized operations, thinking algorithmically
- No computer science degree required



Math & Statistics Knowledge

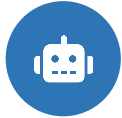
- Needed to extract insight from cleaned data
- Choose and apply appropriate methods
- Baseline: e.g., know what an OLS regression is – and how to interpret it
- No PhD in statistics required



Substantive Expertise

- Science = discovery and building knowledge
- Motivating questions about the world
- Testable hypotheses brought to the data
- Distinguishes data science from pure method application

The Overlaps – and the Danger Zone



Machine Learning (*Hacking + Math & Stats*)

Powerful models and predictions – but without motivating domain questions it is “just” machine learning, not science.



Traditional Research (*Math & Stats + Substantive Expertise*)

Deep methodological and domain knowledge, but little technology – hard to scale to today’s data volumes and sources.

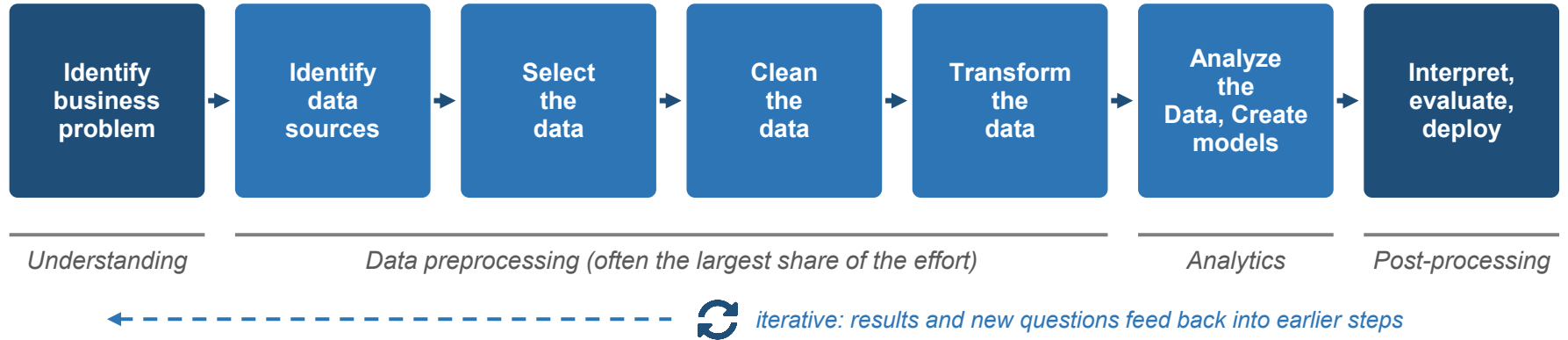


Danger Zone! (*Hacking + Substantive Expertise*)

People who “know enough to be dangerous”: they can extract data and run a regression, but cannot interpret the coefficients – analyses look legitimate but are not understood.

Take-away: competent data science requires all three skill sets – the pairwise overlaps alone are incomplete or even dangerous.

The Data Analytics Process Model



- The model structures every analytics project – from question to deployed solution.
- Garbage in, garbage out: quality of results depends on quality of data → preprocessing dominates the workload.
- Closely related to industry standards such as CRISP-DM (Cross-Industry Standard Process for Data Mining).

cf. Baesens (2014), *Analytics in a Big Data World*

The Process Model in Context of this Module



Identify business problem

Use cases & motivating questions



Identify & manage data sources

Relational data & SQL · NoSQL (MongoDB) · real-time / event-streaming data (MQTT, Apache Kafka)



Select, clean, transform

Python, Jupyter Notebooks · data quality, integration & file formats (XML, JSON)



Analyze

Statistics & machine learning on prepared data



Interpret, evaluate, deploy

Dashboards & web apps (e.g., live map) · decision support · Git/GitHub & Codespaces for delivery

Key Takeaways - Foundations of Data Science

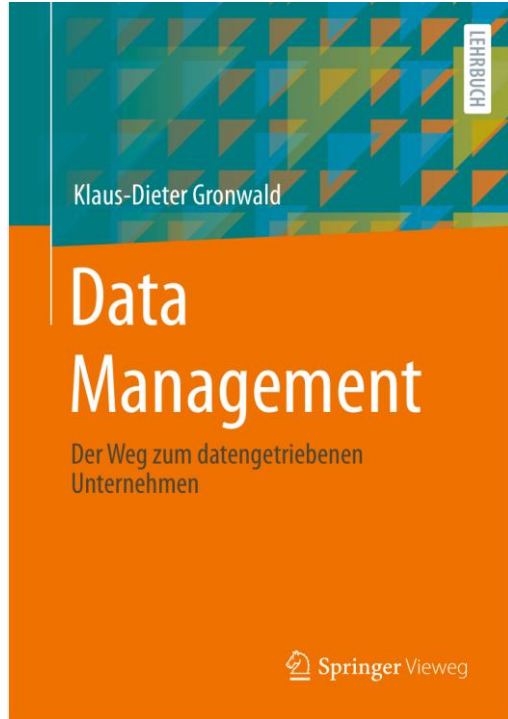
You can ...

- explain what data science is and why it is a multidisciplinary field.
- interpret the Data Science Venn Diagram: the three primary skills, their overlaps, and the Danger Zone.
- explain why all three skill sets are needed for competent data science.
- describe the stages of the Data Analytics Process Model – from business problem to deployment.
- explain why data preprocessing dominates the effort in analytics projects.
- relate the process model to the topics of this module (SQL, NoSQL, real-time data, analytics, deployment).

Recapitulation Management of Relational Data

- Data Governance Framework
- Data Architecture
- Data Modeling
- Database Management
- Enterprise Data Warehouse with Roles
- Data Integration
- Querying Databases using SQL
- Data Quality Management
- Master Data Management
- Metadata Management
- Data Privacy and Compliance
- Data Protection and Security
- Data Lifecycle Management
- Data Archiving
- Data Analysis

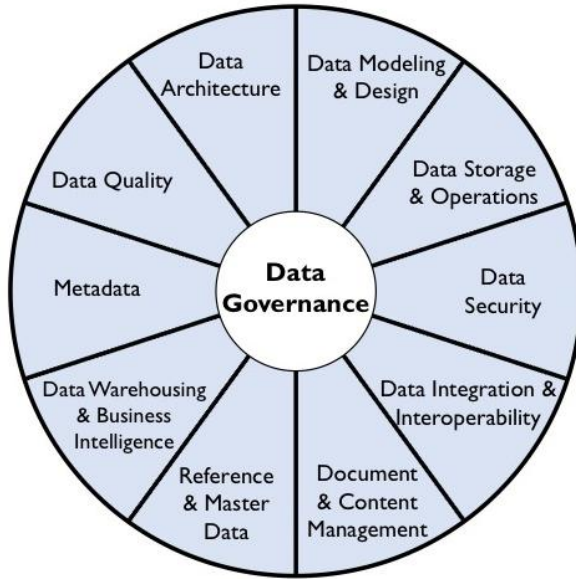
Book Tip - Data Management



“Data management is mainly concerned with how we handle data, how we manage, protect and store it and when we can or must delete which data by setting up rules and creating organizations to implement and comply with these rules.”

Klaus-Dieter Gronwald

DAMA-DMBOK (Data Management Body of Knowledge) Framework



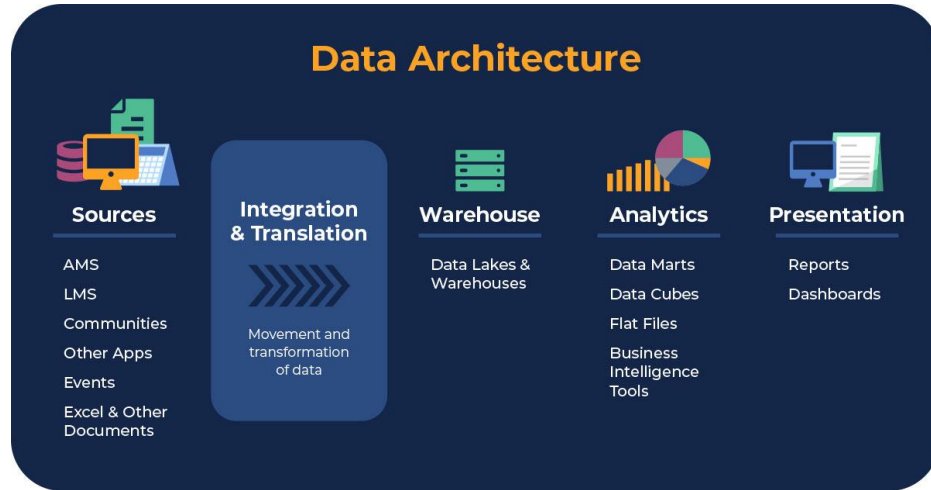
Copyright© 2017 DAMA International

The **DAMA-DMBOK (Data Management Body of Knowledge) Framework** provides a comprehensive and structured approach to data management. It consists of several components, each representing a key area of data management. It places data governance at the center of data management activities, as data governance is required for consistency within and balance between functions.

Key Components: Data governance, Data architecture, Data modeling and design, Data storage and operations, Data security, Data integration and interoperability, Documents and content, Reference and master data, Data warehousing and business intelligence, Metadata, Data quality

Image Credit: <https://www.dama.org/cpages/dmbok-2-wheel-images>

Data Architecture



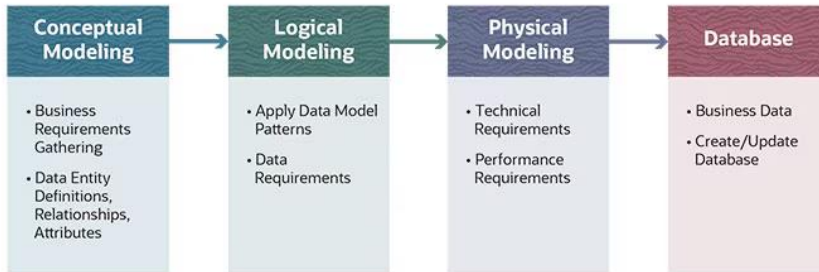
Data architecture describes the conceptual, logical, and physical data assets and how they are stored and managed throughout their lifecycle. It sets the blueprint for data and the way it flows through data storage systems. It is foundational to data processing operations and artificial intelligence (AI) applications.

Key Components: Data Sources, Data Integration, Data Storage, Data Flows, Data Management, Data Analytics, Reporting

Image Credit: <https://cimatri.com/but-we-have-the-data-the-importance-of-data-architecture>

Data Modeling - Overview and Model Levels

Data Modeling Process Workflow

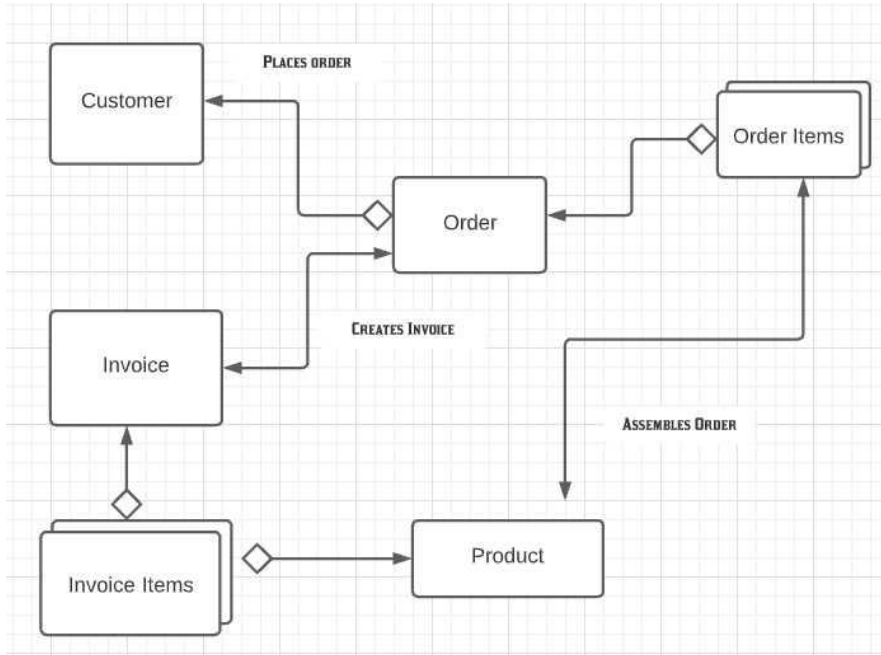


Data modeling is the process of researching, analyzing and determining the scope of data requirements. The scope of the data requirements is presented and communicated in a data model. This process is iterative and includes conceptual, logical and physical models.

Key Data Models: Conceptual Data Model, Logical Data Model, Physical Data Model

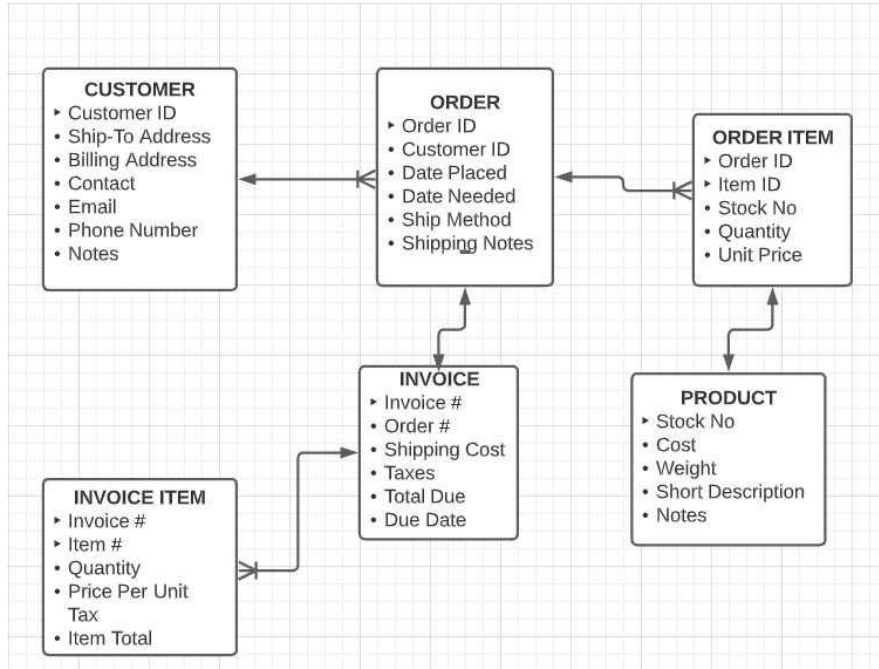
Image Credit: <https://www.netsuite.com/portal/resource/articles/data-warehouse/data-modeling.shtml>

Data Modeling - Conceptual Model



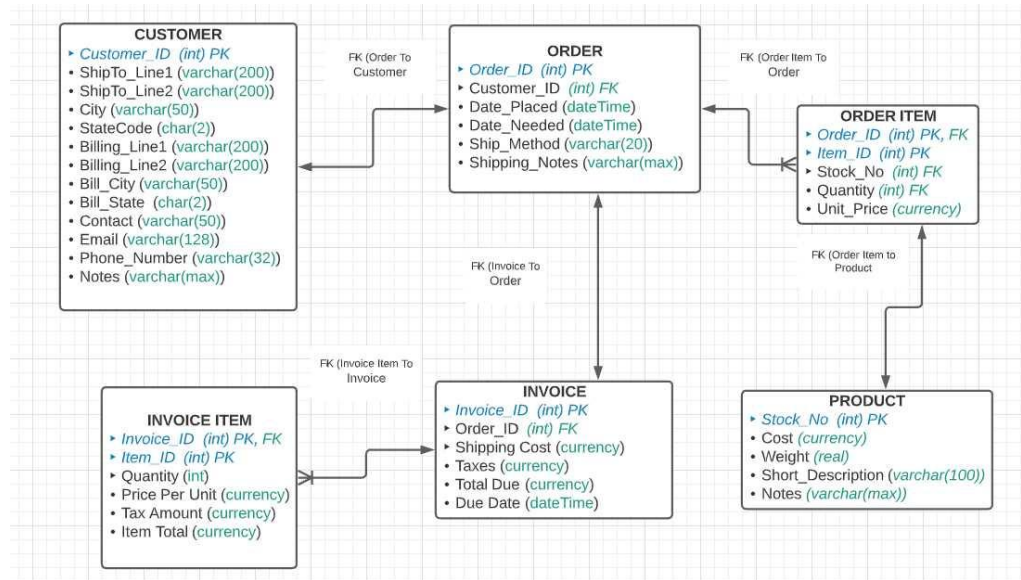
The aim of a **conceptual data model** is to provide a data-centric perspective of the organization by documenting how different business entities relate to one another. It is the most abstract data model that describes the data elements and their relations without detail. Unlike logical and physical data models, conceptual data models are technology- and application-independent.

Data Modeling - Logical Model



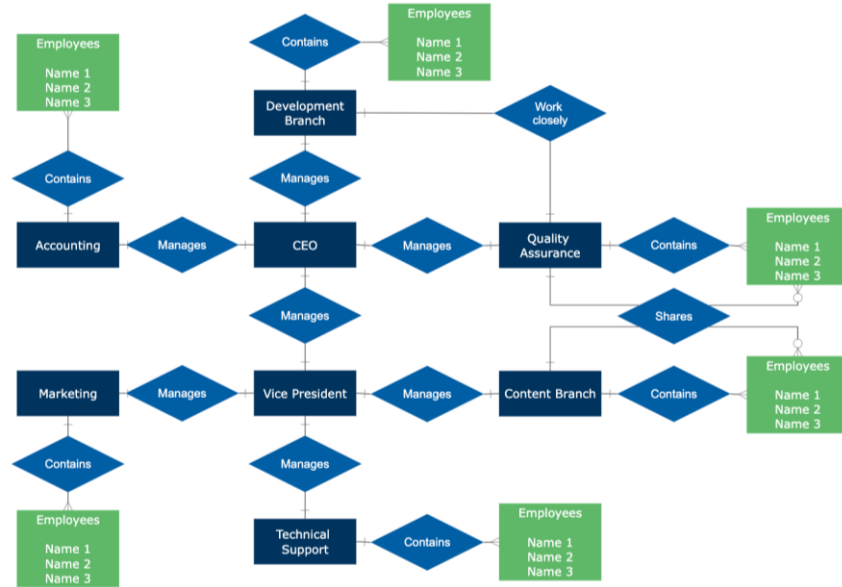
A **logical data model** is a model that provides a detailed, structured description of data elements and the connections between them. It includes all entities — a specific object transferred from the real world (relevant to business) — and the relationships among them. These entities have defined their attributes as their characteristics.

Data Modeling - Physical Model



A **physical data model** specifies how the data model will be built in the database. It outlines all table structures, including column name, data types, column constraints, primary key and foreign key with indexes to the relevant table column, relationships between tables, stored procedures, and views.

Data Modeling - Key Concepts

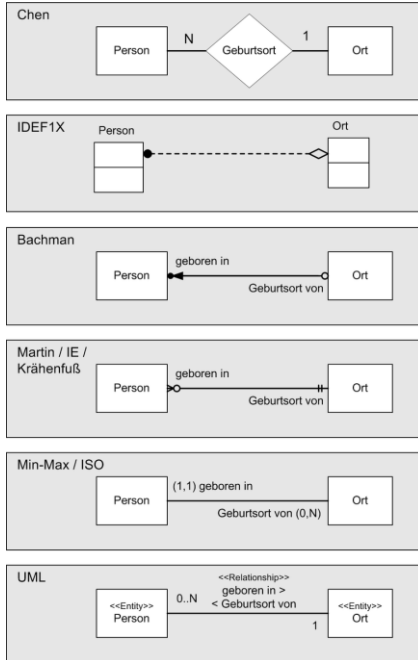


Key Concepts of Data Modeling:

- **Entities and Attributes:** Entities represent objects or concepts, and attributes are the properties or details about these entities.
- **Relationships:** Connections between entities, often represented as one-to-one, one-to-many, or many-to-many relationships.
- **Normalization:** The process of organizing data to reduce redundancy and improve data integrity.
- **Data Integrity Constraints:** Rules that ensure the accuracy and consistency of the data, such as primary keys, foreign keys, and unique constraints.

Image Credit: <https://www.smartdraw.com/entity-relationship-diagram>

Data Modeling - ER Diagram Notations

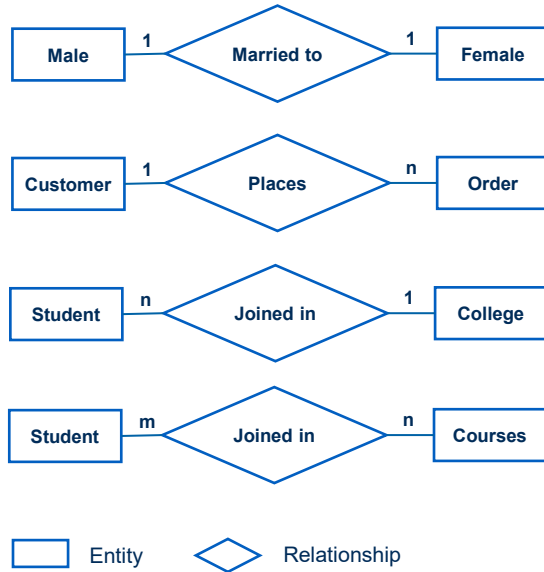


Notation forms of **Entity Relationship (ER) diagrams**:

- **Chen notation** by Peter Chen, the developer of ER diagrams, 1976; extended by the representation of attributes using Modified Chen Notation (MC notation).
- **IDEF1X**, the de facto standard used by US authorities for many years.
- **Bachman notation** by Charles Bachman as a widely used tool diagram language.
- **Martin notation** (crow's foot notation) as a widely used tool diagram language (information engineering).
- **(min, max) notation** by Jean-Raymond Abrial, 1974.
- **UML** as a standard that even ISO uses in its own standards as a replacement for ER diagrams.

Image Credit: <https://de.wikipedia.org/wiki/Entity-Relationship-Modell>

Data Modeling - Relationship Cardinalities



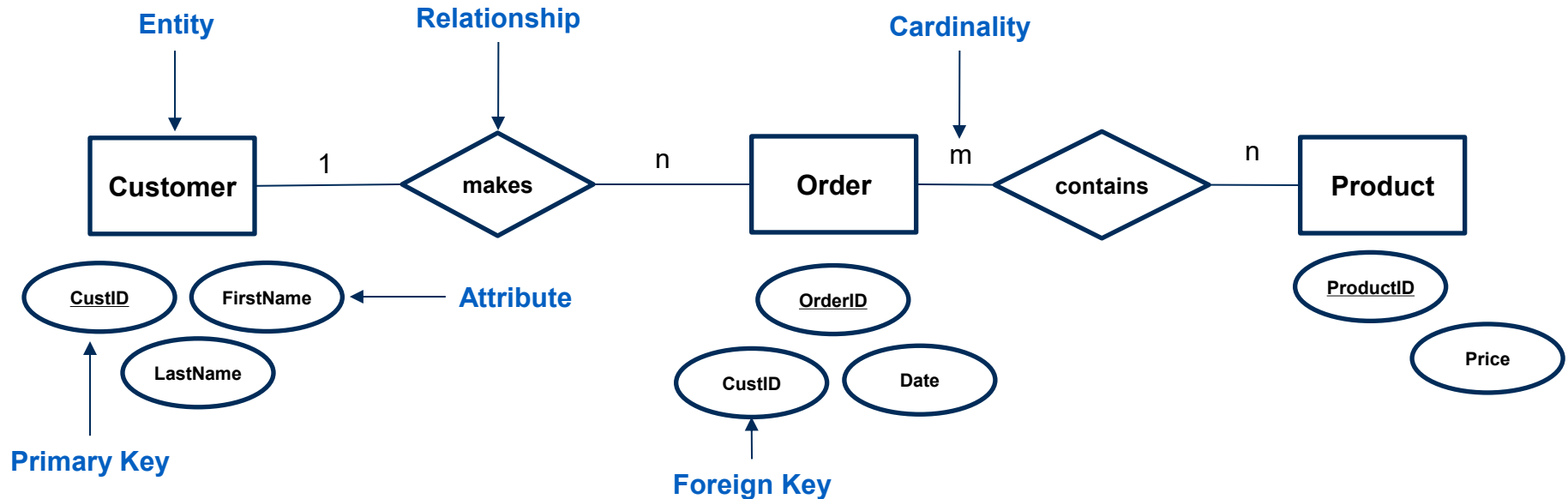
In a Relational Database Management System (RDBMS), **relationships** define how entities (tables) are associated with each other. The main types of **cardinalities** (which describe the relationships between entities) in a DBMS are:

- one-to-one (1:1)
- one-to-many (1:n)
- many-to-one (n:1)
- many-to-many (m:n)

Image Credit: <https://prepinsta.com/dbms/relationship-cardinality/>

Data Modeling - Example ER Diagram

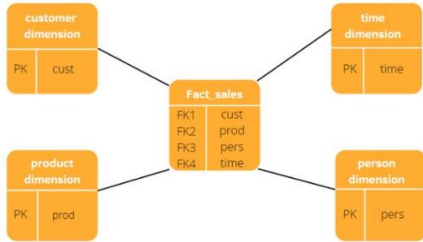
Example ER Diagram



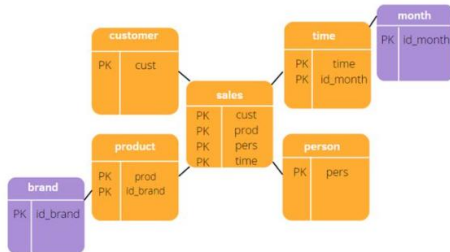
Source: [ER diagram tutorial on YouTube](#)

Data Modeling - Star and Snowflake Schemas

Star Schema



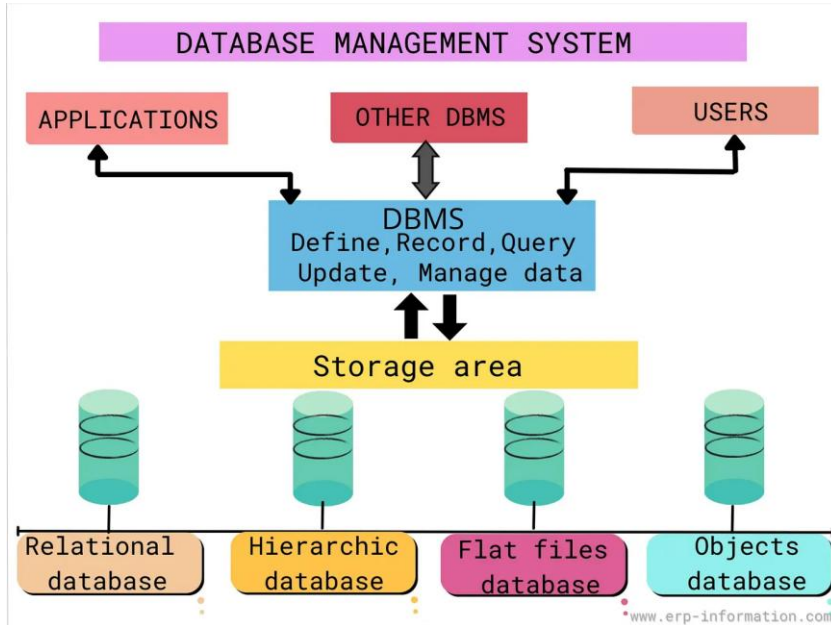
Snowflake Schema



Star and snowflake schemas are database designs used in data warehousing. A **star schema** features a central fact table (e.g. sales amounts) connected directly to dimension tables (e.g. product details), creating a simple, star-like structure that is easy to query but may have data redundancy. In contrast, a **snowflake schema** normalizes these dimension tables into multiple related tables, forming a more complex snowflake pattern that reduces redundancy and storage needs but requires more complex queries and can be slower.

Image Credit: <https://in.pinterest.com/pin/star-schema-vs-snowflake-schema-for-data-warehouse--773704411004255617/>

Database Management - Overview and Components



Database Management refers to the processes and tools used to control, maintain, and operate databases. It encompasses the creation, deployment, and management of databases to ensure they operate efficiently, securely, and reliably.

Key Components: Database Management Systems (DBMS), Data Security, Backup and Recovery, Performance Tuning, Data Integrity, Scalability

Image Source: <https://salemal.medium.com/what-is-a-database-system-72c831e0e057>

Database Management - RDBMS Examples



PostgreSQL



ORACLE®
DATABASE



Relational Database Management Systems

(RDBMS) are software systems used to manage relational databases, where data is stored in tables with rows and columns.

Examples of RDBMS are:

- MySQL
- PostgreSQL
- Microsoft SQL Server
- Oracle Database
- IBM Db2
- MariaDB
- Amazon Aurora
- SQLite

Database Management - Docker and pgAdmin Configuration

docker-compose.yml (see GitHub repository on slide 34)

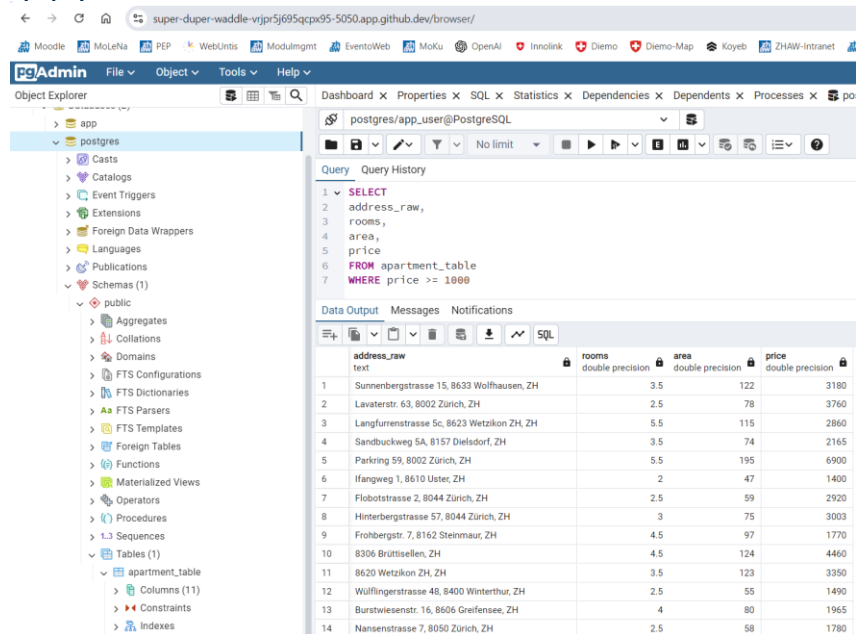
```
1  services:
2
3  app:
4    build: .
5    volumes:
6      - ./workspace:cached
7    command: sleep infinity
8    network_mode: service:db
9    depends_on:
10     - db
11
12  pgadmin:
13    image: dpape/pgadmin4:latest
14    restart: always
15    environment:
16      PGADMIN_DEFAULT_EMAIL: pgadmin@gmail.com
17      PGADMIN_DEFAULT_PASSWORD: geheim
18      PGADMIN_CONFIG_WTF_CSRF_HEADERS: '["Referer", "Origin"]'
19      PGADMIN_CONFIG_WTF_CSRF_ENABLED: 'False'
20  volumes:
21    - pgadmin-data:/var/lib/pgadmin
22  ports:
23    - 5050:80
24  depends_on:
25    - db
26
27  db:
28    image: postgres:latest
29    restart: always
30    environment:
31      POSTGRES_DB: postgres
32      POSTGRES_USER: pgadmin
33      POSTGRES_PASSWORD: geheim
34    volumes:
35      - pg-data:/var/lib/postgresql/data
36
37  volumes:
38    pgadmin-data:
39    pg-data:
```

Settings for pgAdmin database management tool

Settings for PostgreSQL database

Database Management - Administration Tools

pgAdmin for the administration of PostgreSQL



Relational Database Management Systems (RDBMS) often come with **database management software** designed to help users interact with, manage, and administer relational databases. These tools provide a user-friendly interface to perform various tasks related to database management without requiring extensive knowledge of SQL (Structured Query Language).

Database Management - PostgreSQL Exercise

Exercise: Querying data from a PostgreSQL database with Python and SQL

GitHub repository: [Open the PostgreSQL exercise repository](#)

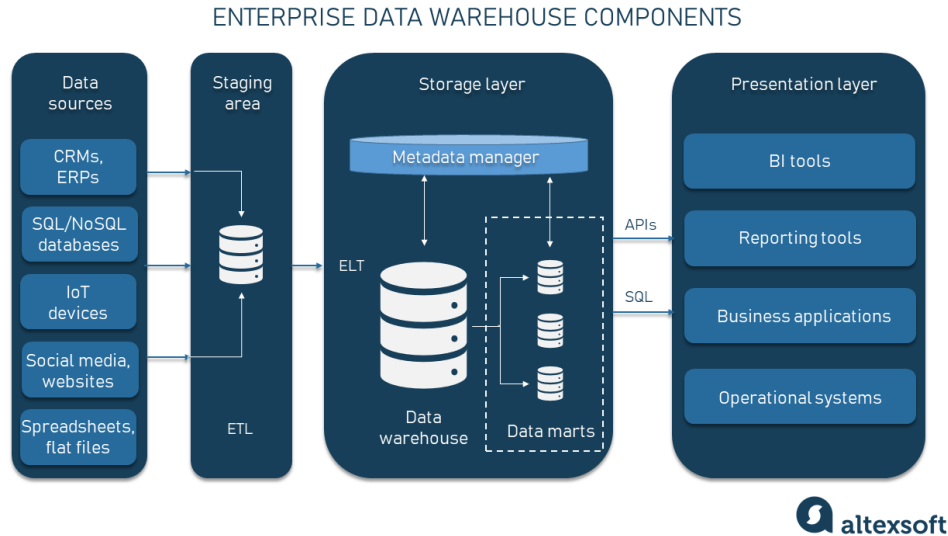
Steps:

- Fork the repository into your own GitHub account.
- Create a new GitHub Codespaces environment based on the repository.

Tasks (will be shown in class):

1. Run the Jupyter notebook and try to understand what the Python code does.
2. Answer the questions provided in SQL/QUESTIONS.sql.

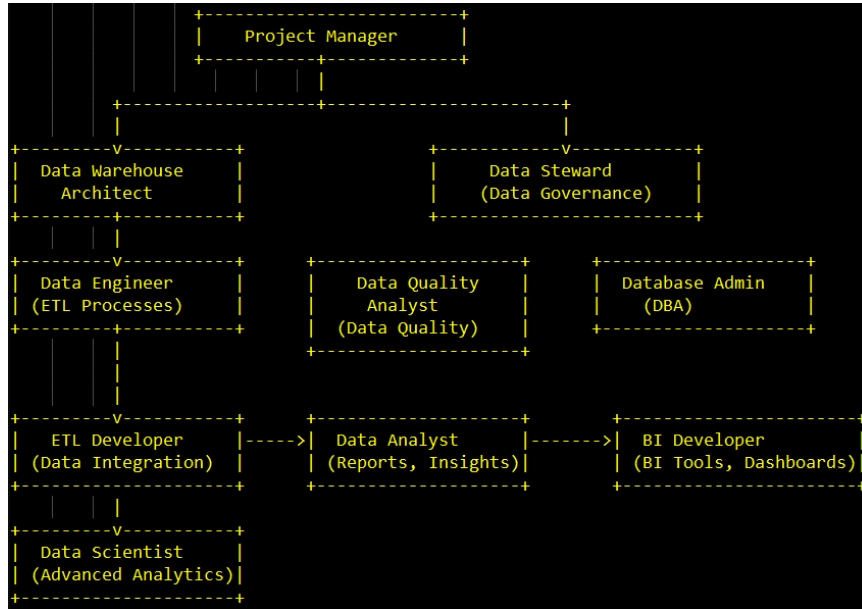
Enterprise Data Warehouse - Concepts



A **Data Warehouse (DWH)** is a centralized repository designed to store, manage, and analyze large volumes of data from multiple sources. It is structured to support business intelligence (BI) activities, particularly data analytics and reporting. The data stored in a data warehouse is typically cleaned, transformed, and organized in a way that makes it easy to query and analyze.

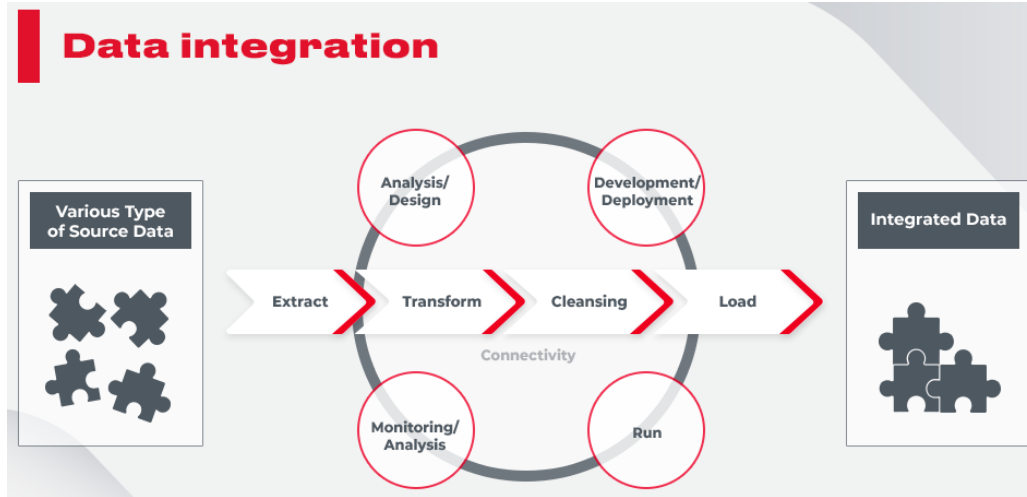
Image Credit: <https://www.altexsoft.com/blog/enterprise-data-warehouse-concepts/>

Enterprise Data Warehouse - Roles



- **Project Manager** oversees the entire data warehouse project.
- **Data Warehouse Architect** designs the system, working under the Project Manager.
- **Data Engineer** and **ETL Developer** handle data processing and integration under the Architect.
- **Database Administrator** manages the database and collaborates with the Architect.
- **Data Quality Analyst** and **Data Steward** ensure data quality and governance, interfacing with various roles.
- **Data Analyst** and **BI Developer** focus on analysis and visualization.
- **Data Scientist** conducts advanced analytics, often collaborating with Data Analysts.

Data Integration



Data Integration is the process of combining data from different sources to provide a unified view. It involves extracting, transforming, and loading data (ETL) from disparate systems into a single repository, making it accessible and usable for analysis and decision-making.

Key Components: ETL Process, Data Sources, Data Quality, Data Mapping, Real-Time Integration

Image Credit: <https://estuary.dev/benefits-of-data-integration/>

Data Types

Common SQL Data Types:

1. Numeric:

- `INT`: Integer values.
- `FLOAT`: Floating-point numbers.
- `DECIMAL(p, s)`: Fixed-point numbers with precision `p` and scale `s`.

2. String:

- `CHAR(n)`: Fixed-length string of `n` characters.
- `VARCHAR(n)`: Variable-length string up to `n` characters.
- `TEXT`: Large text data.

3. Date/Time:

- `DATE`: Date values.
- `TIME`: Time values.
- `DATETIME`: Date and time combined.

4. Boolean:

- `BOOLEAN`: `TRUE` or `FALSE` values.

5. Binary:

- `BLOB`: Large binary objects (e.g., images).

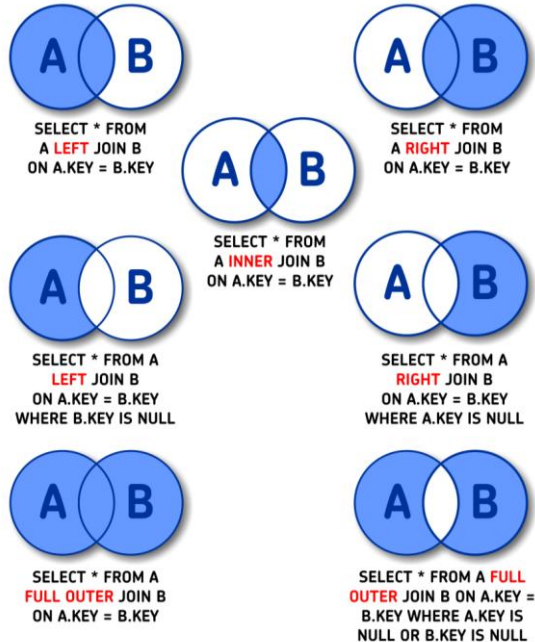
In SQL, a **data type** defines the kind of value that can be stored in a particular column of a table. It specifies the nature of the data, such as whether it is numeric, textual, date/time-related, or something else. Each column in an SQL table must have a defined data type, which determines the operations that can be performed on the data and the amount of storage space required.

Querying Databases with SQL - Overview and Example

```
SELECT
    e.EmployeeID,
    e.FirstName,
    e.LastName,
    e.Salary,
    d.DepartmentName,
    d.ManagerID
FROM
    Employee e
LEFT JOIN
    Department d
    ON e.Department = d.DepartmentName
WHERE
    e.Salary > (
        SELECT
            AVG(Salary)
        FROM
            Employee
        WHERE
            Department = e.Department)
ORDER BY e.LastName;
```

SQL (Structured Query Language) is a standardized programming language specifically designed for managing and manipulating relational databases. It is used to perform various operations on the data stored in a database, such as querying, updating, inserting, and deleting data, as well as creating and modifying database structures like tables and views.

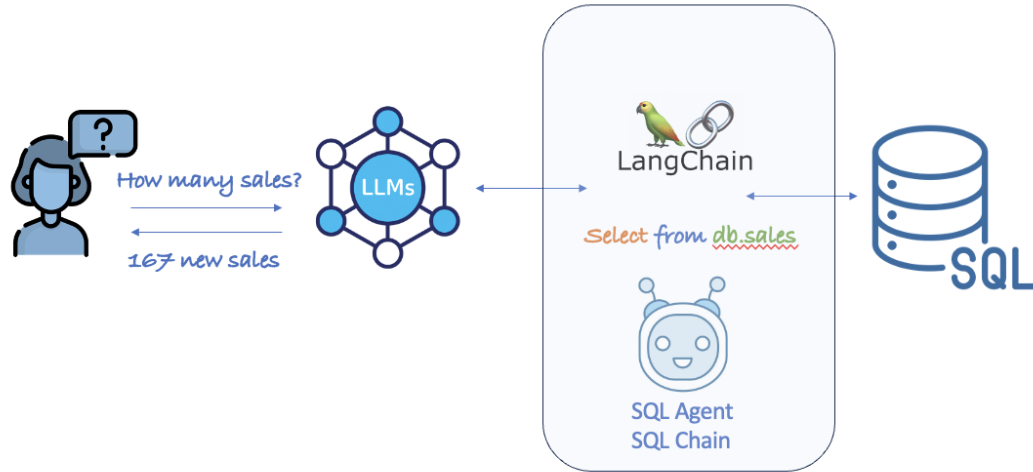
Querying Databases with SQL - Joins



SQL joins are operations in relational databases that allow you to combine rows from two or more tables based on a related column between them.

Image Credit: <https://medium.com/@aakriti.sharma18/joins-in-sql-4e2933cedde6>

SQL Databases with AI Agents - Concept



With the advancement of artificial intelligence (AI), new opportunities have emerged to automate complex tasks, including database queries. The **LangChain SQL Agent**, which can be used with various large language models, enables database queries to be formulated in natural language. This significantly simplifies access to data and reduces the need for specialized SQL knowledge.

Image Credit: <https://community.sap.com>

SQL Databases with AI Agents - Exercise

Exercise: Interacting with SQL Databases Using LangChain's SQL Agents

GitHub repository: [Open the LangChain SQL Agent exercise repository](#)

Steps:

- Fork the repository into your own GitHub account.
- Create a new GitHub Codespaces environment based on the repository.

Tasks:

1. Run the example and try to understand the code.
2. Create additional queries of the data.

Data Quality Management



Image Source:

<https://www.starfishetl.com/blog/data-quality-management-what-it-and-how-do-it>

Data Quality Management (DQM) refers to the processes, policies, and technologies used to ensure that an organization's data is accurate, consistent, complete, and reliable. It involves continuous monitoring, maintenance, and improvement of data quality to support effective decision-making and operational efficiency.

Key Components: Data Profiling, Data Rules, Data Monitoring, Data Remediation, Data Quality Reporting, Data Discovery

Master Data Management

First steps in a master data management initiative

Input from domain specialists helps ensure data definitions make sense in practical, day-to-day, real-world cases.

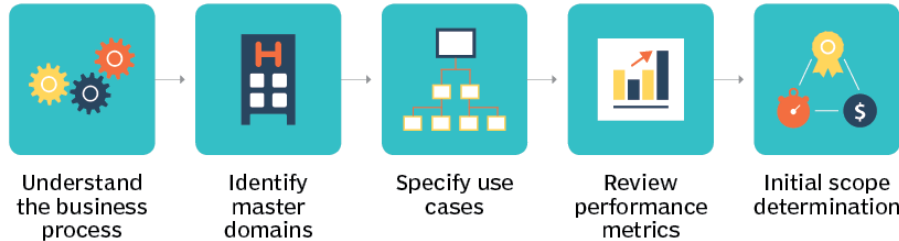


Image Credit:

<https://www.techtarget.com/searchdatamanagement/definition/master-data-management>

Master Data Management (MDM) refers to a set of processes, governance, policies, standards, and tools that define and manage the critical data of an organization to provide a single point of reference.

In practice, it involves creating and maintaining a consistent, accurate, and authoritative view of master data across the enterprise, ensuring that this data is shared and used by different parts of the organization consistently. This master data often includes key information about customers, products, employees, and suppliers.

Metadata Management

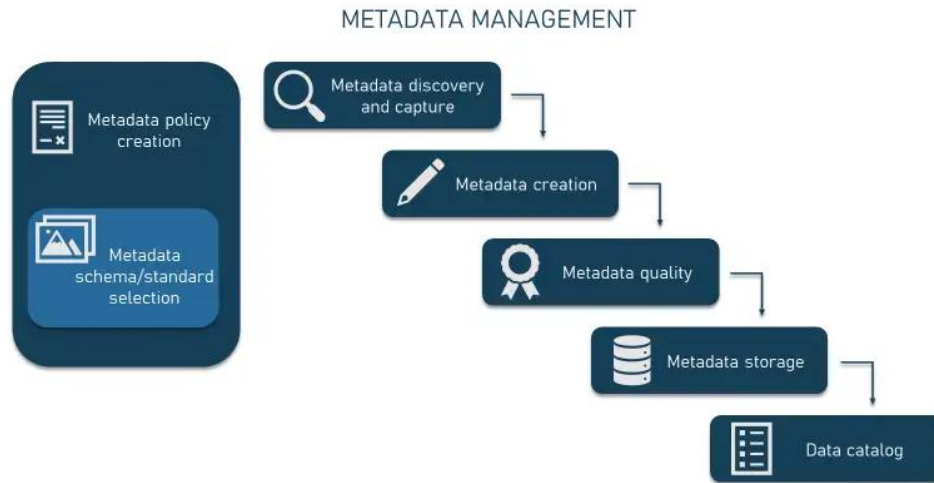


Image Credit: <https://www.altexsoft.com/blog/metadata-management>

Metadata Management involves the administration of data that describes other data, known as metadata. It includes the creation, storage, and control of metadata to ensure that data is well-defined, easily discoverable, and usable across the organization.

Key Components: Metadata Catalog, Data Lineage, Metadata Standards, Metadata Governance, Business and Technical Metadata

Data Privacy and Compliance



Image Credit: <https://www.planetcrust.com>

Data Privacy and Compliance refers to the practices, laws, and regulations that organizations must follow to protect the personal data they collect, process, and store. This includes ensuring that data is collected legally, used responsibly, and stored securely to prevent unauthorized access, breaches, or misuse. Compliance involves adhering to specific legal requirements, such as the General Data Protection Regulation (GDPR) in Europe.

Key Components: Consent, Transparency, Security, Access, Accuracy, Retention, Accountability, Compliance, Breach-Notification, Data-Minimization

Data Protection and Security

Data Protection

Traditional Data Protection			Data Security			Data Privacy		
Backup/Restore	RAID & Erasure Coding	Replication	Encryption	Threat Monitoring	Authentication	Legislation	Policies	Best Practice
Archiving	Physical Infrastructure	Data Retention	Access Control	Breach Access and recovery	Data Loss Prevention	3 rd party contacts	Data Governance	Global variations

Timely
availability
of
unspoiled
data

Data Protection and Security

involve the practices and technologies designed to safeguard data from unauthorized access, corruption, or theft, while ensuring its confidentiality, integrity, and availability.

Key Components: Data Encryption, Access Control, Data Masking, Data Backup and Recovery, Compliance Management, Security Audits

Image Source: <https://www.snia.org/education/what-is-data-privacy>

Data Lifecycle Management



Image Credit: <https://www.clicdata.com/blog/complete-guide-data-lifecycle-management/>

Data Lifecycle Management (DLM) refers to the policies, processes, and practices involved in managing data throughout its entire lifecycle, from creation to deletion. The concept is crucial for ensuring that data is handled efficiently, securely, and in compliance with relevant regulations.

Key Components: Data Creation, Data Storage, Data Usage, Data Archival, Data Destruction

Data Archiving

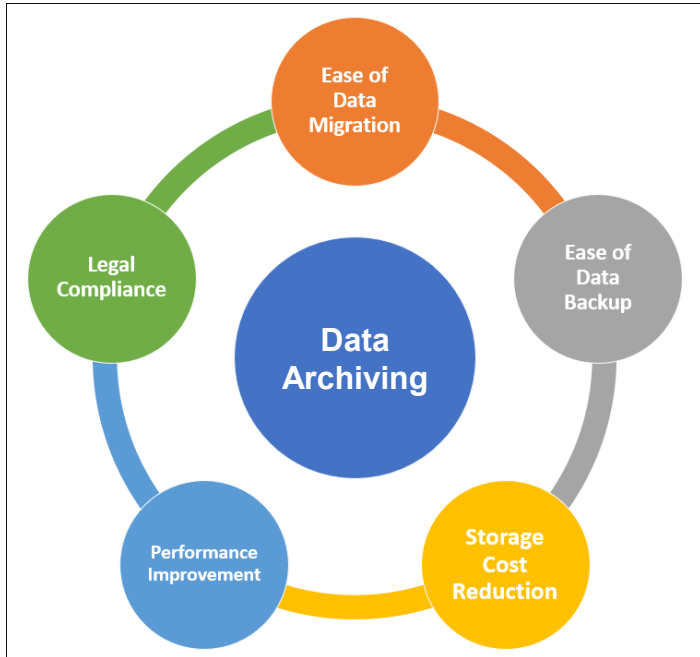
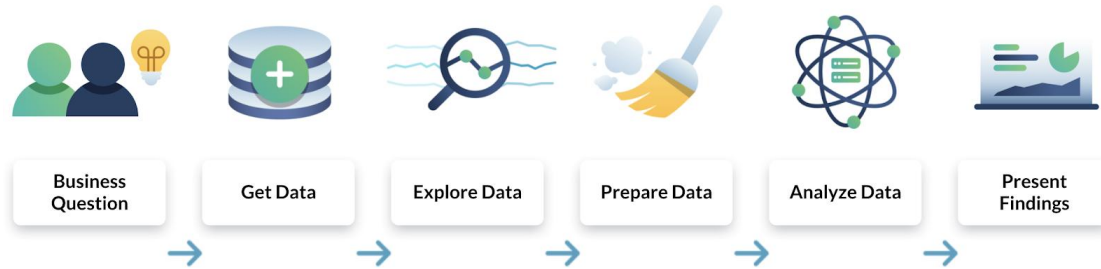


Image Credit: <https://community.sap.com>

Data Archiving refers to the process of moving data that is no longer actively used to a separate storage system for long-term retention. It ensures that important historical data is preserved and remains accessible when needed, while reducing the burden on primary storage systems.

Key Components: Performance Improvement, Ease of Data Migration, Data Backup, Storage Cost Reduction, Legal Compliance

Data Analysis - Process and Components



Data Analysis is the process of examining, cleaning, transforming, and modeling data with the goal of discovering useful information, drawing conclusions, and supporting decision-making.

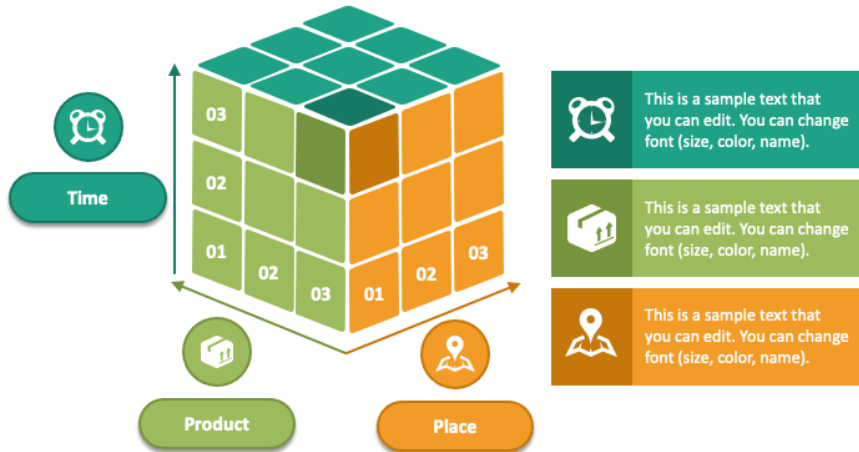
Key Components: Data Preparation, Exploratory Data Analysis (EDA), Statistical Hypothesis Testing, Predictive Modeling, Presentation of Results

Image Credit: <https://www.datacamp.com/blog/what-is-data-analysis-expert-guide>

Data Analysis - OLAP

OLAP CUBE

Enter your sub headline here



OLAP (Online Analytical Processing) is a technology used for performing complex data analysis and queries on large volumes of data stored in databases, typically within a data warehouse. It allows users to interactively analyze multidimensional data from multiple perspectives, such as sales data by region, time period, and product category.

Key Components: Dimensions, Hierarchies, Levels, Measures, Facts, Aggregates, Cells, Slices, Dicing, Drill Down/Drill Up.

Key Takeaways - Relational Data Management

You can explain ...

- what a data governance framework includes
- the role of data architecture
- the creation and application of different data models
- the processes involved in managing databases
- what a data warehouse is, and which roles can be found there
- what data integration means, and the types of data used in databases
- how to write SQL queries
- what data management entails
- the importance of managing key organizational (master) data
- the organization and control of metadata
- practices for implementing data privacy and adhering to regulations
- methods to protect data from breaches and unauthorized access
- the lifecycle management of data from creation to destruction
- techniques for analyzing data to gain insights, including the OLAP method

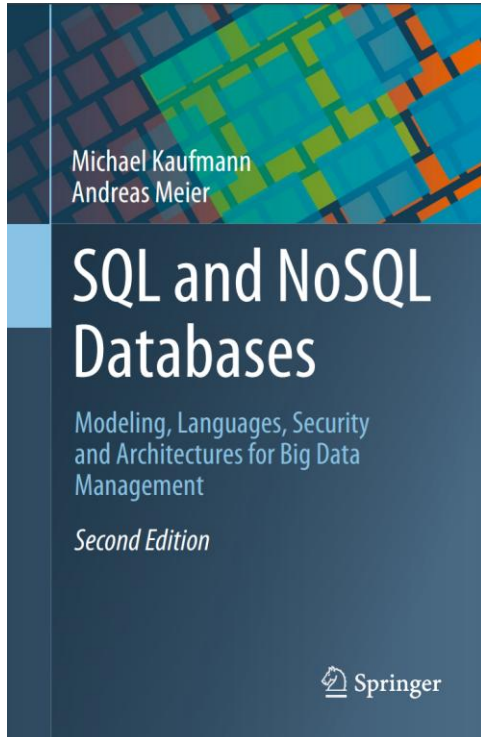
Literature - Relational Data Management

- Gronwald, K. D. (2024). Data Management: Der Weg zum datengetriebenen Unternehmen. Springer-Verlag.
- Henderson, D., Early, S. (Eds.) (2017). DAMA-DMBOK: Data management body of knowledge. Technics Publications, LLC.
- SQL Tutorial (n.d.). <https://www.tutorialspoint.com/sql/index.htm>

Recapitulation Management of Non-Relational (NoSQL) Data

- Book Tip
- What is a Non-Relational Database and why is it useful?
- Common Types of NoSQL Databases
 - Key-Value Databases
 - Column-based Databases
 - Graph Databases
 - Document-oriented Databases
- Querying Collections using MongoDB Query Language (MQL)

Book Tip - SQL and NoSQL Databases



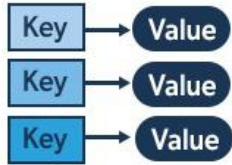
“Nevertheless, the developments in the Big Data ecosystem brought new technologies into the world of databases, to which we pay enough attention too. Non-relational database technologies, which find more and more fields of application under the generic term NoSQL, differ not only superficially from the classical relational databases but also in the underlying principles.”

Michael Kaufmann, Andreas Meier

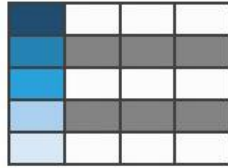
What is a NoSQL Database and why is it useful?

Common Types of NoSQL Databases

Key-Value

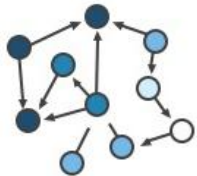


Column-Family



NoSQL (short for "Not Only SQL") refers to a category of databases that are designed to handle large volumes of unstructured, semi-structured, or structured data. Unlike traditional relational databases (RDBMS), which use structured tables with predefined schemas, NoSQL databases are more flexible and can manage a wide variety of data types.

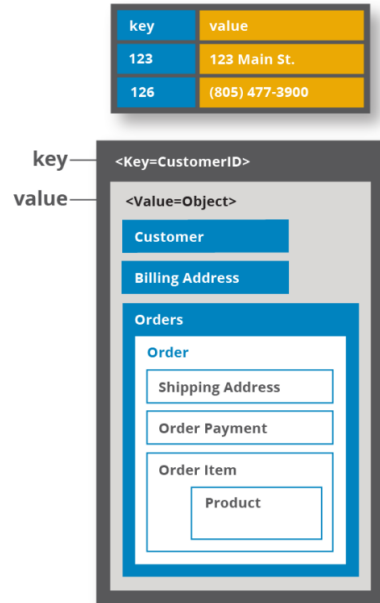
Graph



Document



Common Types of NoSQL Databases: Key-Value Databases



Key-Value databases are a type of NoSQL database that store data as a collection of key-value pairs. They are designed to handle massive amounts of data with high speed and scalability.

Image Credit: <https://hazelcast.com/glossary/key-value-store/>

Common Types of NoSQL Databases: Column-based Databases

ID	Last name	First name	Bonus
53666	Smith	James	8000
50333	Jones	Sam	5000
54673	Taylor	Ann	4000
58930	Burton	Sue	9000

Row-oriented

53666	Smith	James	8000
50333	Jones	Sam	5000
54673	Taylor	Ann	4000
58930	Burton	Sue	9000

Column-oriented

53666	50333	54673	58930
Smith	Jones	Taylor	Burton
James	Sam	Ann	Sue
8000	5000	4000	9000

Column-based databases, also known as column-oriented databases, store data by columns rather than by rows. This layout is used by analytical (columnar) databases; wide-column NoSQL databases such as Cassandra or HBase group columns into column families. It is especially advantageous in analytics and big data environments.

Common Types of NoSQL Databases: Graph Databases

Graph-Model (E-Commerce)

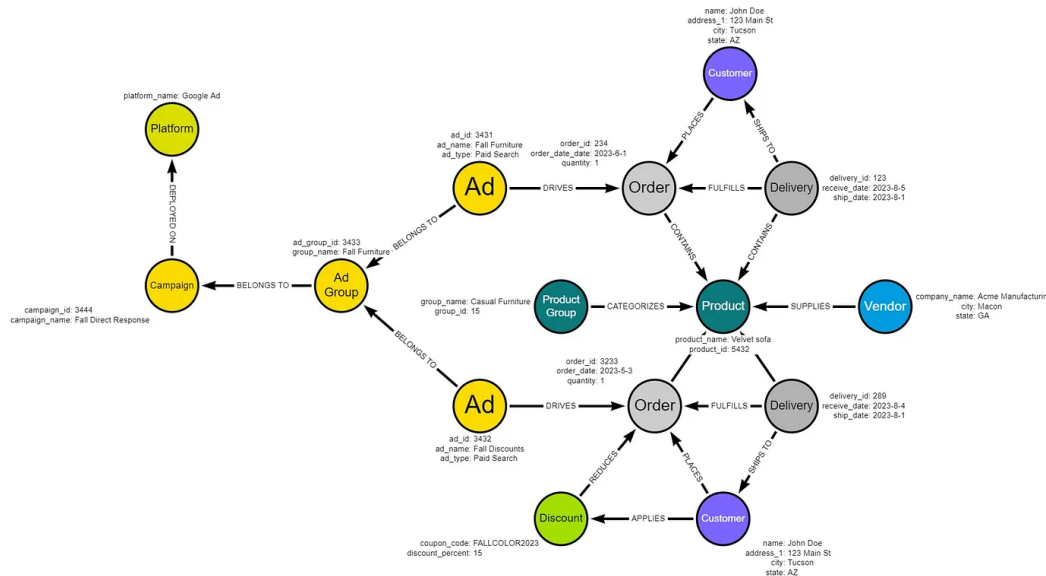
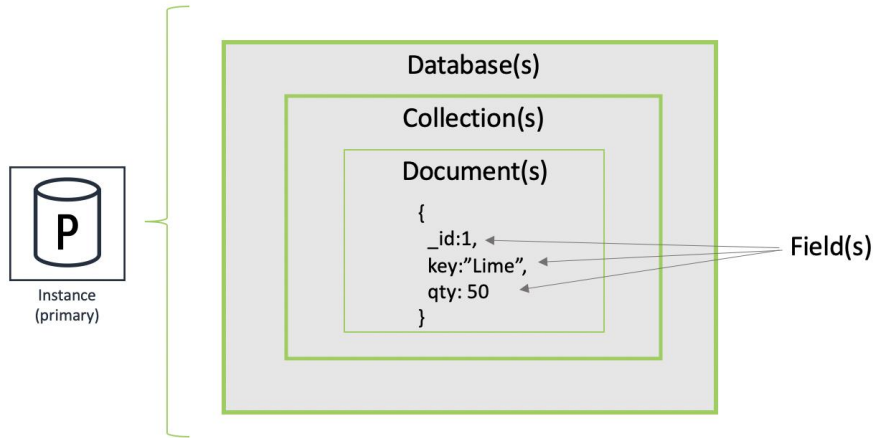


Image Credit: <https://towardsdatascience.com/why-you-need-a-knowledgegraph-and-how-to-build-it-ac4f35cb75b7>

Graph databases are a type of NoSQL database that represent and store data in terms of nodes, edges, and properties. This structure is highly efficient for modeling and querying complex relationships between data, making graph databases particularly useful in scenarios where connections between entities are as important as the entities themselves.

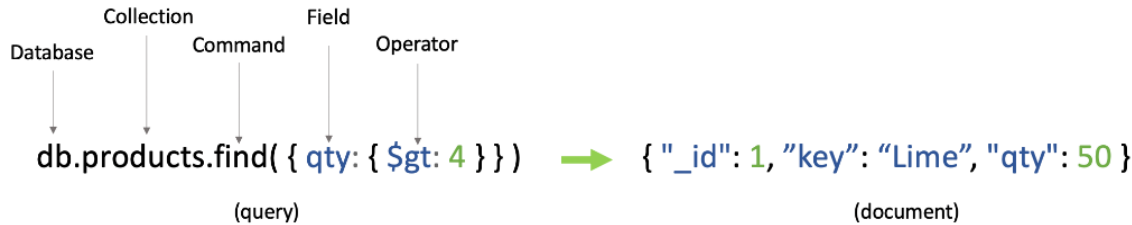
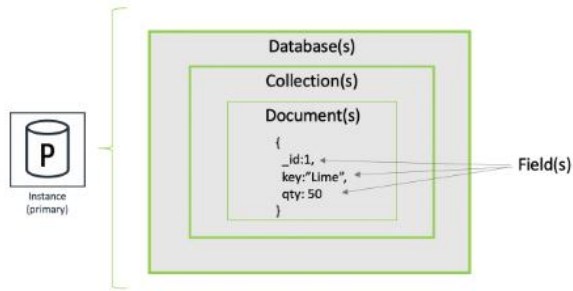
Document-Oriented Databases - MongoDB Documents



A **document database** (like MongoDB) is a type of NoSQL database that can be used to store and query data as JSON-like documents.

Developers can use JSON documents in their code and save them directly in the document database. The flexible, semi-structured and hierarchical nature of documents and document databases allows them to evolve with the requirements of the applications.

Document-Oriented Databases - MongoDB Query Language



MongoDB Query Language (MQL) is the way to interact with MongoDB databases to perform operations like querying, updating, and managing data stored in MongoDB collections.

Document-Oriented Databases - Sharding

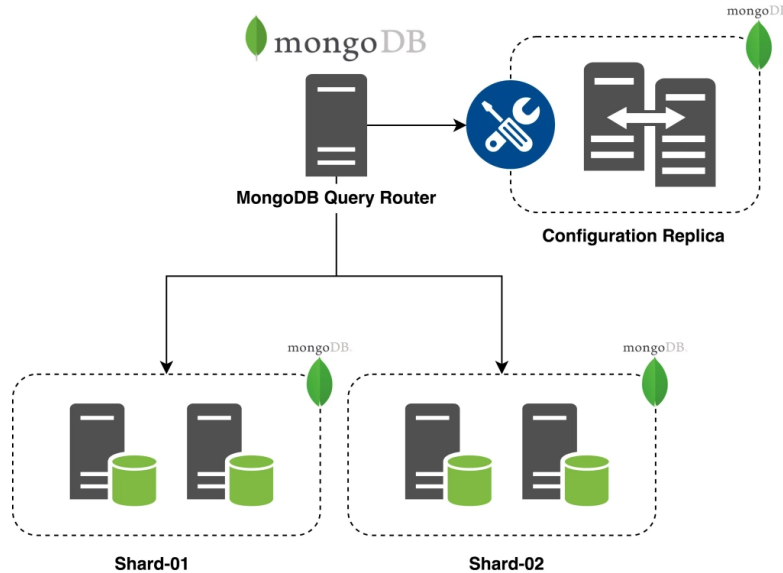


Image Credit: <https://madurad.wordpress.com/2018/11/06/shard-cluster-deployment-configuration-mongodb/>

Sharding in MongoDB is a method used to distribute data across multiple machines to ensure scalability and high availability. A shard in MongoDB is a horizontal partition of data in a sharded cluster, where each shard contains a subset of the data to distribute load and improve performance. It is particularly useful when dealing with large datasets or high throughput operations that exceed the capacity of a single server.

MongoDB Query Language - Mongo Playground Example

A screenshot of the Mongo Playground web interface. The browser address bar shows "mongoplayground.net". The interface has a dark header with navigation links like "Template", "single collection", "run", "format", and "share". Below the header, there are two main panels. The left panel, labeled "Database", shows a JSON document for a restaurant named "Zum Alten Löwen" with fields for type, id, location, tags, and detailed address information. The right panel, labeled "Query", shows a MongoDB query: `1- db.collection.find({ 2- "tags.cuisine": "italian,pizza" 3- })`.

[Open Mongo Playground](#)

For example data see 'restaurants_zuerich.txt' data of this week

MongoDB Query Language - Exercise

Exercise: Querying data from a MongoDB with Python and MQL

GitHub repository: [Open the MongoDB exercise repository](#)

Steps:

- Fork the repository into your own GitHub account.
- Create a new GitHub Codespaces environment based on the repository.

Tasks (will be shown in class):

1. Run the examples in the Jupyter Notebooks and try to understand the code.
2. Create at least 5 additional MQL queries for both car and restaurant data.

Key Takeaways - NoSQL Data Management

You can ...

- explain what a non-relational database is and why it is useful.
- explain the different types of NoSQL databases.
- explain the characteristics of key-value databases.
- explain the structure and benefits of column-based databases.
- explain how graph databases store and manage data relationships.
- explain document-oriented databases and their applications.
- query collections using MongoDB Query Language (MQL).

Literature - NoSQL Data Management

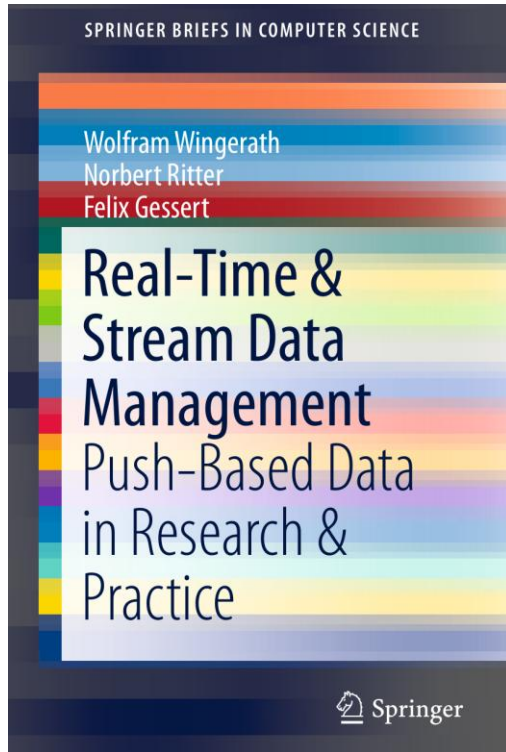
- Kaufmann, M., & Meier, A. (2023). *SQL and NoSQL Databases*. Springer Cham.
- MongoDB Tutorial (n.d.). <https://www.tutorialspoint.com/mongodb/index.htm>

Management & Use of Real-time Data

Part I: Basics of Real-time Data & Event Streaming

- Book Tip
- What is Real-time Data?
- Why Real-time Data Matters
- Applications of Real-time Data
- Introduction to Event Streaming
- Stream processing architecture
- Publisher-Subscriber Model

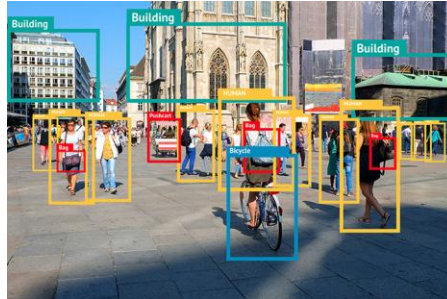
Book Tip



“Within the scope of this book, however, real-time databases are systems that provide push-based access to database collections. Likewise, real-time queries in the context of this book are push-based queries on top of database collections.”

Wolfram Wingerath, Norbert Ritter, Felix Gessert

What is Real-time Data?



Real-time data is information that is delivered and processed immediately after collection. It allows for instantaneous decision-making without delays. Examples of real-time data include live social media feeds, stock market prices, and sensor readings in IoT devices.

Why Real-time Data Matters



Enhanced Decision-Making: Organizations can make informed decisions quickly based on the latest data.

Improved Operational Efficiency: Enables automation and immediate responses to events, reducing downtime and improving productivity.

Competitive Advantage: Businesses can respond faster to market changes, customer needs, and emerging risks.

Applications of Event Streaming

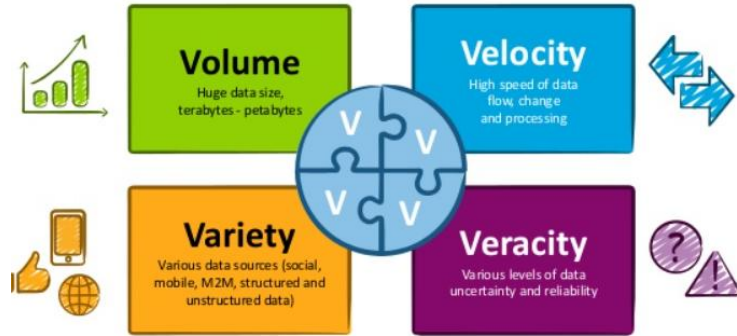
Applications of event streaming & real-time analytics

- Patient monitoring in healthcare
- Algorithmic trading
- Online fraud detection
- Geofencing
- Location tracking
- Traffic management
- Product recommendations
- Churn detection
- IoT sensor data analytics
- Social media data analytics
- Gaming data feed

Introduction to Event Streaming - Paradigm Shift

The paradigm shift from traditional data warehouses to streaming platforms

Properties of Big Data



Traditional BI-Infrastructure (more or less «static»)

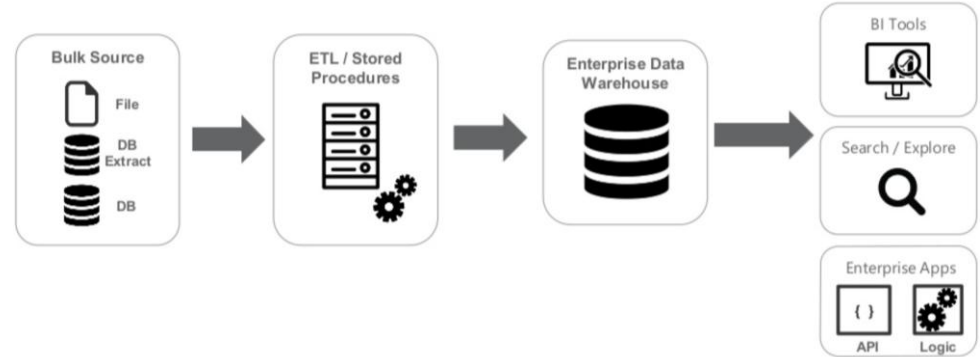
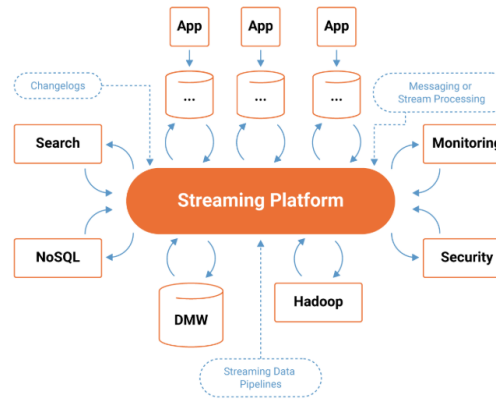
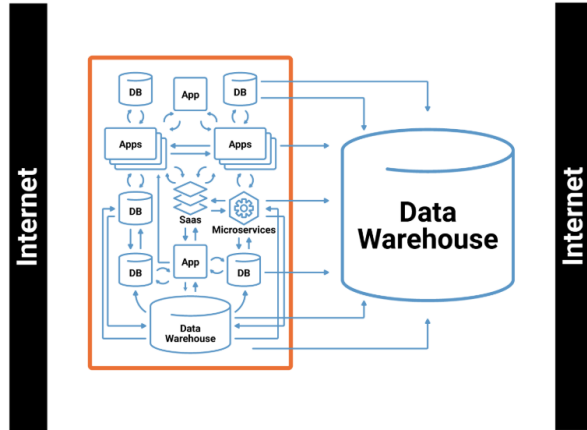


Image Credit: Guido Schmutz (2019) Introduction to Stream Processing

Introduction to Event Streaming - Limits of Traditional BI

The paradigm shift from traditional data warehouses to streaming platforms

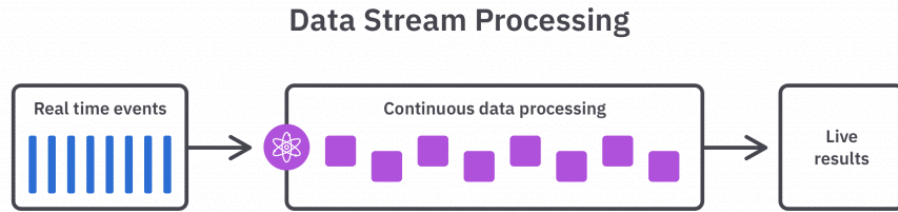


Due to the velocity of big data, traditional (high-latency) BI infrastructure reaches its limits. This is where event streaming comes in.

Image Credit: <https://www.bankingclub.de/news/it/event-streaming-und-der-weg-zu-ruhm-und-reichtum/>

Introduction to Event Streaming - Definitions

What is Event Streaming and Stream Processing?

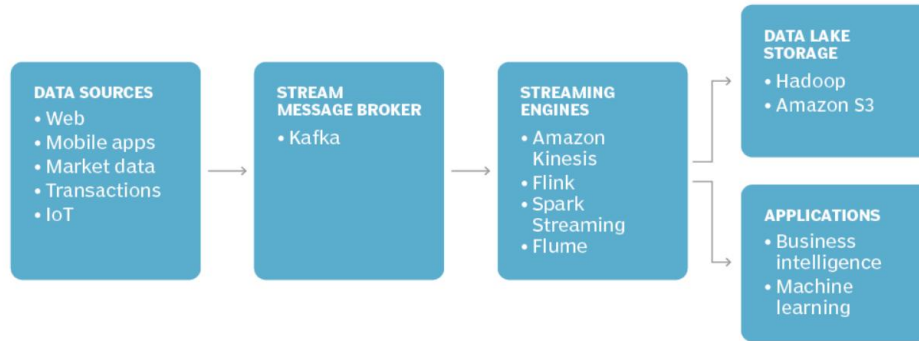


The term **event** refers to each data point in the system, and **stream** refers to the ongoing delivery of those events. A series of events can also be referred to as **streaming data** or **data streams**.

Image Credit: <https://estuary.dev/streaming-data-processing>

Stream-Processing Architecture - Components

Stream processing basic architecture

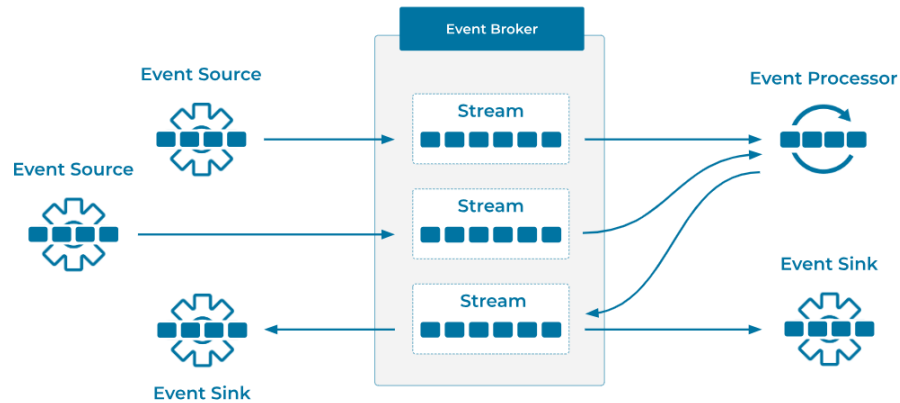


Components of a streaming architecture:

- Data Sources
- Stream Message Broker
- Streaming Engine
- Storage
- Applications

Image Credit: <https://searchdatamanagement.techtarget.com/feature/How-to-build-an-effective-streaming-data-architecture>

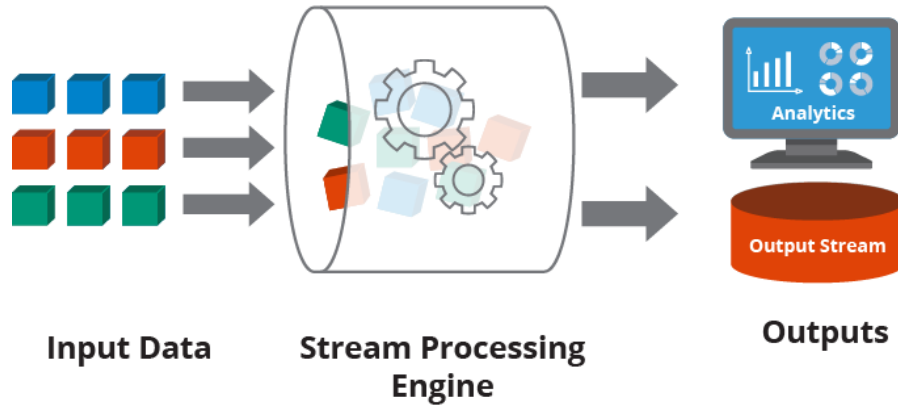
Stream-Processing Architecture - Message Broker



An **Event (also called Stream-) Message Broker** is a critical component in event-driven architectures, particularly in the context of event streaming. It acts as an intermediary that facilitates the flow of data (in the form of events or messages) between producers (data sources) and consumers (data processors or services).

Image Credit: <https://developer.confluent.io/patterns/event-stream/event-broker/>

Stream-Processing Architecture - Processing Engine



A **Stream Processing Engine** is software designed to process and analyze data streams in real-time. These engines are integral to event streaming architectures, enabling the ingestion, processing, and output of continuous data flows as they are generated. Stream Processing Engines are built to handle large volumes of data with low latency, ensuring that insights can be derived from data in near real-time.

Image Credit: <https://blog.devgenius.io/event-processing-platform-4d950c4ff3e3>

Stream-Processing Architecture - Data Storage

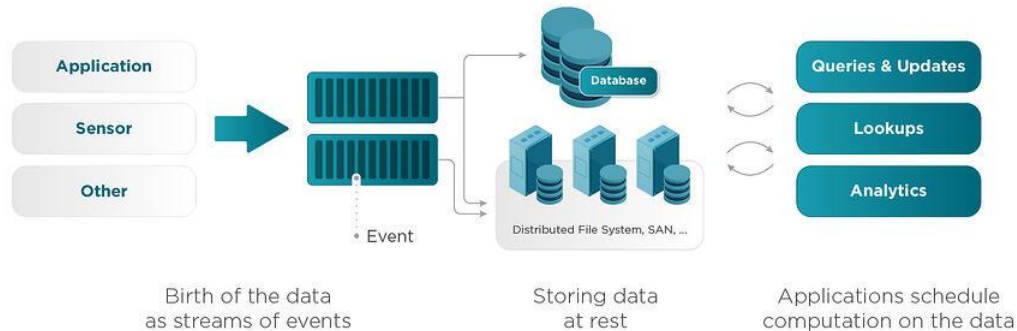
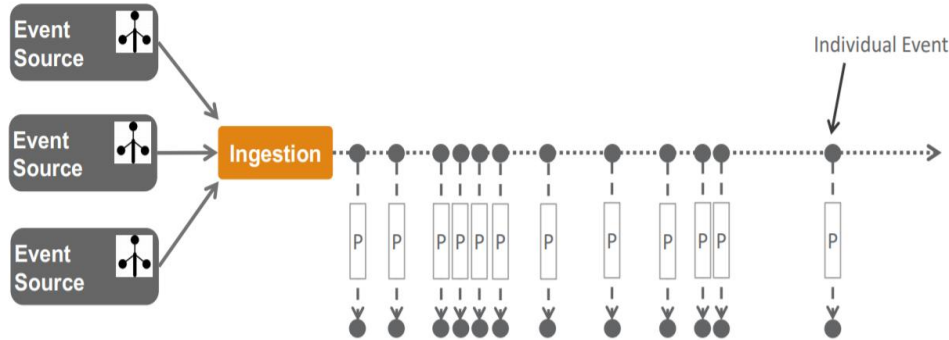


Image Credit: <https://www.ververica.com/what-is-stream-processing>

Streaming Data Storage refers to the mechanisms and systems used to store, manage, and retain data that is generated and ingested in a continuous, real-time manner. In most streaming platforms, persistent data storage is also available, allowing for long-term retention and retrieval of streaming data even after it has been processed.

Stream-Processing Architecture - Continuous Streaming

Streaming Model – Continuous Streaming

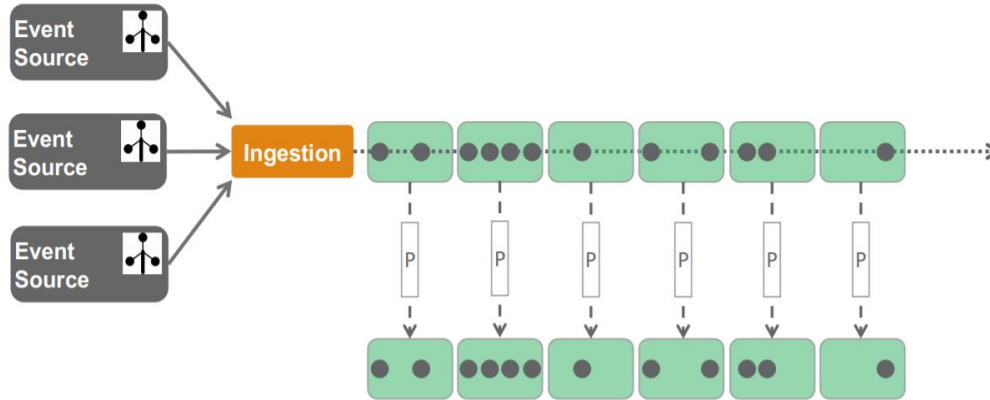


- Events processed as they arrive
- Low-latency
- Less throughput
- Fault tolerance expensive

Image Credit: Guido Schmutz (n.d.) Introduction to Streaming Analytics
See: <https://vimeo.com/289696402>

Stream-Processing Architecture - Micro-Batching

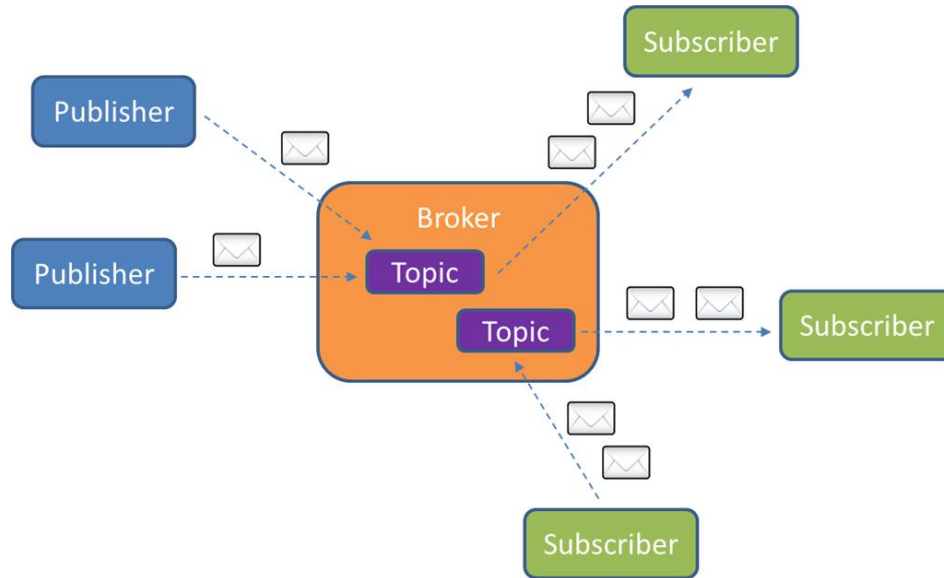
Streaming Model – Micro-Batch



- Splits incoming stream into small batches
- Could provide higher throughput
- Fault tolerance easier to achieve
- Higher latency

Image Credit: Guido Schmutz (n.d.) Introduction to Streaming Analytics
See: <https://vimeo.com/289696402>

Publisher-Subscriber Model - Overview



In a **publisher / subscriber model**, a client can be the publisher or subscriber or both. A publisher publishes a message on a topic. A subscriber subscribes to a topic and receives all messages published on that topic. The topic is the mechanism by which clients exchange messages. The broker manages all subscriptions and relays published messages.

Image Credit:

http://www.embedded101.com/Portals/0/images/DNNArticle/Windows-Live-Writer/77907af8df9c_DF61/Fig3.3_4.png

Publisher-Subscriber Model - Kafka Concepts

Record, Topic, Partition, Log, Offset

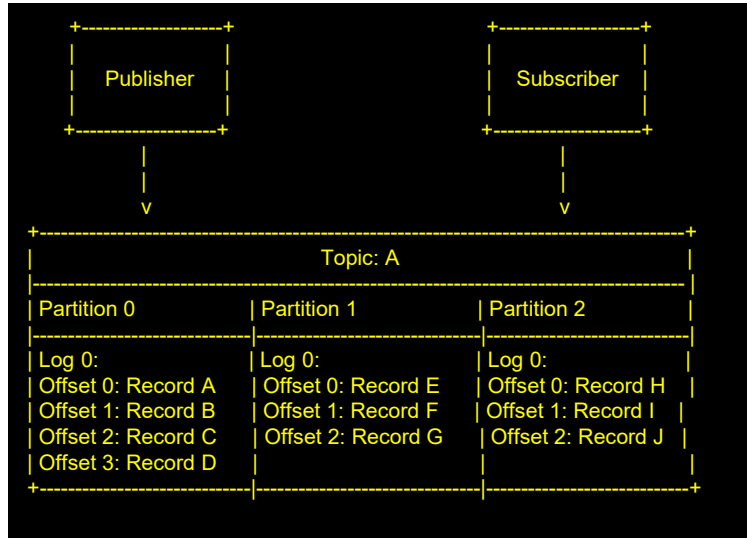


Image Credit: https://www.alibabacloud.com/blog/an-overview-of-kafka-distributed-message-system_594218

- **Event records** (also called messages) are byte arrays that can store objects such as JSON documents. Event records are organized into topics.
- A **topic** is a logical grouping or category where messages are published and subscribed to. It stores, routes, and facilitates the sending and receiving of event records.
- A topic can be divided into multiple **partitions**.
- Event records are retained in distributed **logs**.
- The position of an event record in the log is called **offset**.

Publisher-Subscriber Model - Kafka Example

Record, Topic, Partition, Log, Offset



Example: The Publisher sends records to the Topic "A". The Topic "A" has three Partitions (Partition 0, Partition 1, Partition 2). Each Partition contains a Log of records, with each record having a unique Offset. The Subscriber reads from one or more partitions, typically keeping track of the last offset it has read to continue processing from that point onward.

Key Takeaways - Real-Time Data and Event Streaming

You can ...

- define and describe real-time data.
- explain why real-time data is crucial.
- identify key use cases of real-time data.
- explain the components of a stream-processing architecture
- explain the fundamentals of event streaming.
- explain the publisher-subscriber model.

Literature - Real-Time Data and Event Streaming

Wingerath, W., Gessert, F., Friedrich, S., & Ritter, N. (2016). Real-time stream processing for Big Data. *it-Information Technology*, 58(4), 186-194.

Management & Use of Real-time Data

Part II: Event Streaming with MQTT and Apache Kafka

- **The MQTT Protocol (originally MQ Telemetry Transport)**
 - Introduction to MQTT
 - MQTT Applications
 - MQTT Architecture
 - MQTT Protocol Details
 - MQTT Security
 - MQTT Implementations and Ecosystem
 - MQTT Example: Streaming IoT Sensor Data in Real-Time
- **Apache Kafka**
 - Introduction to Apache Kafka
 - Apache Kafka Architecture
 - Message Flow and Topics
 - Kafka Clustering and Scaling
 - Key Kafka Operations and Tools
 - Apache Kafka Example: Live Bus Routes in the City of Zürich

Introduction to MQTT

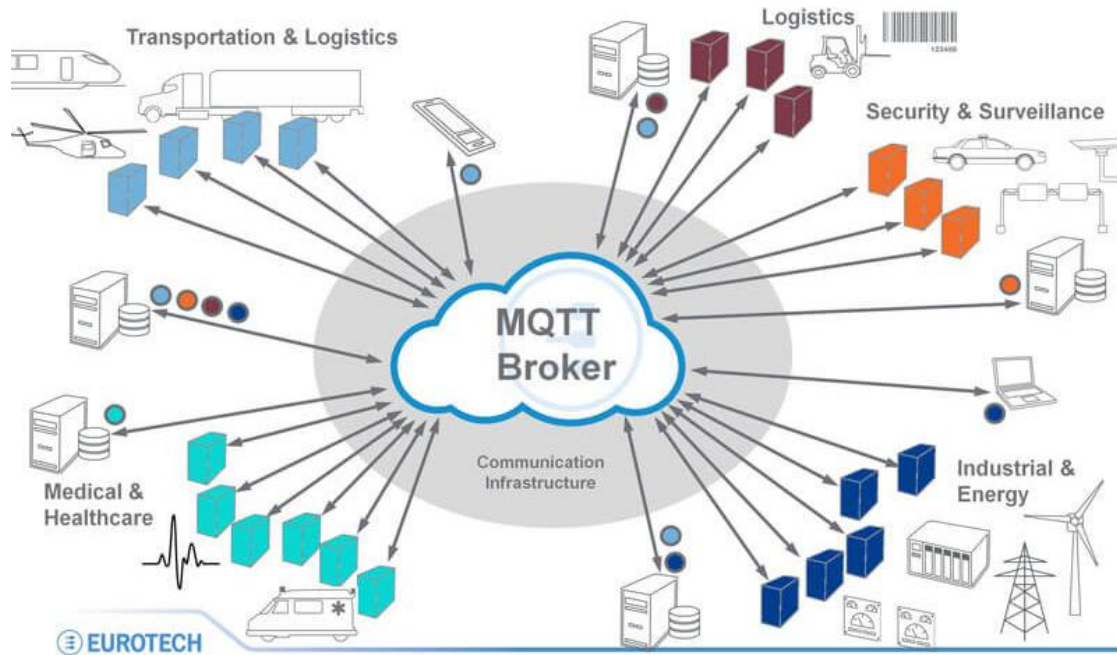
What is the MQTT Protocol (originally MQ Telemetry Transport)?

- Event-driven publish / subscribe architecture
- The publisher and subscriber connect to a broker
- Messages are sent over topics
- Wildcard subscriptions allow the reception of specific messages on topics
- Quality of Service (QoS) allows the usage of three different service qualities
- Last will allows the definition of predefined messages, if the client disconnects



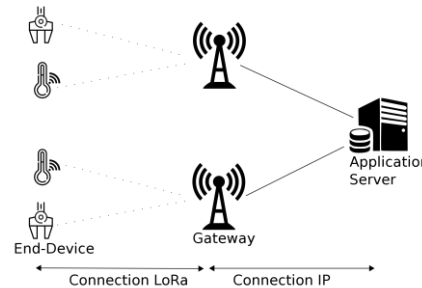
<https://mqtt.org>

MQTT Applications - Overview



MQTT is one of the most widely used protocols for **Machine-to-Machine (M2M)** and **Internet of Things (IoT)** applications. Based on the publisher / subscriber architecture, it simplifies the connectivity between devices.

MQTT Applications - LoRaWAN



MQTT and **Long Range Wide Area Network (LoRaWAN)** are often combined in IoT solutions: LoRaWAN connects low-power devices over long distances to gateways, and the network server typically forwards the data to applications via MQTT. Typical applications are smart agriculture, environmental monitoring and smart cities.

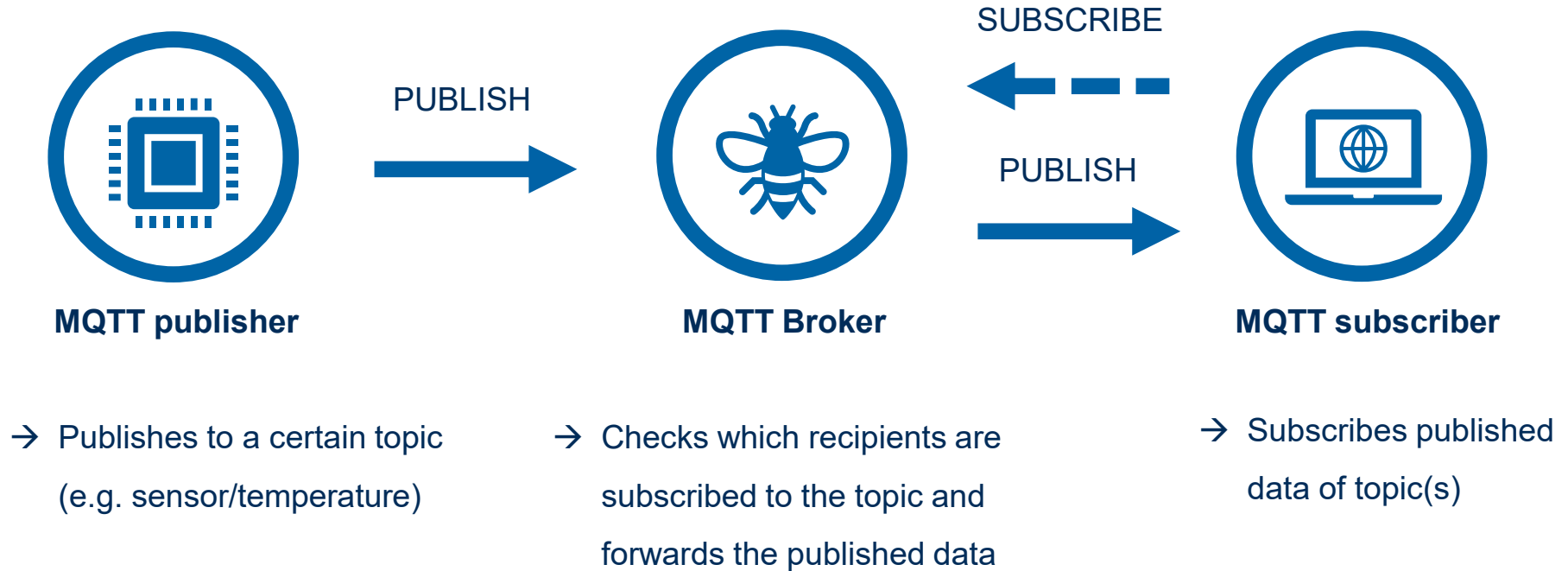
Image Credit

https://de.wikipedia.org/wiki/Long_Range_Wide_Area_Network

LoRaWAN video on YouTube

<https://www.mouser.ch/new/stmicroelectronics/stm-steval-strkt01-lora-iot-tracker/>

MQTT Architecture



MQTT Protocol Details

QoS: Quality of Service

QoS 0: At most once



QoS 1: At least once



QoS 2: Exactly once



MQTT Protocol Details

Message Structure:

- Header: Contains control flags and packet type.
- Payload: Carries the actual message data.

Quality of Service (QoS) Levels:

QoS 0: At most once delivery (fire and forget).

QoS 1: At least once delivery (with acknowledgment).

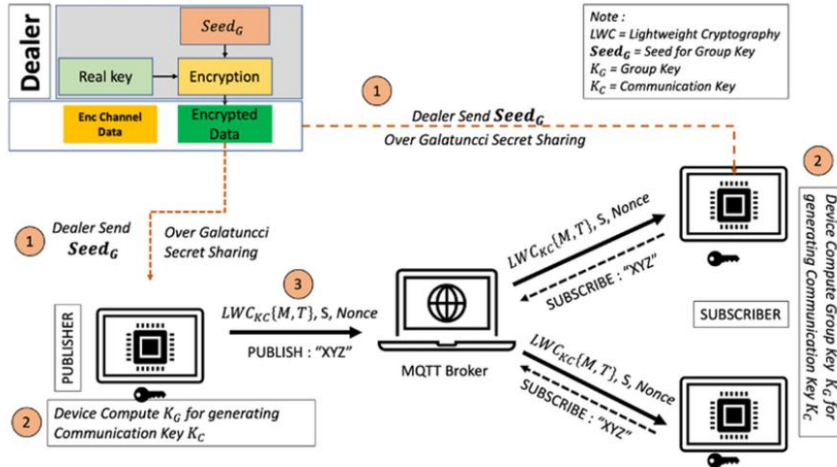
QoS 2: Exactly once delivery (involves a four-step handshake).

Retained Messages: Allows MQTT to store the last message on a topic so that new subscribers immediately receive the latest data.

Last Will and Testament (LWT): A message sent when a client disconnects unexpectedly, useful for detecting failures.

Image Credit: <https://datenberg.eu/ressourcen/wissen/mqtt-in-der-industrie/>

MQTT Security



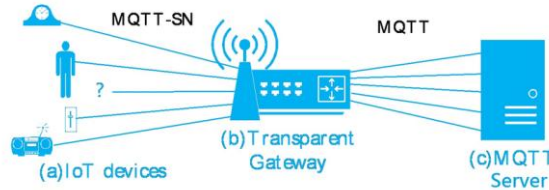
Secure End-to-End Design on MQTT Protocol.

Image Credit: https://www.researchgate.net/figure/Secure-End-to-End-Design-on-MQTT-Protocol_fig2_363220338

MQTT Security Overview

- **Transport Security:** MQTT can utilize TLS/SSL protocols to encrypt messages, ensuring secure communication between clients and brokers.
- **Authentication:** Username and Password & Client Certificates
- **Authorization:** Brokers can enforce topic-level permissions, allowing or restricting access to specific topics for different clients.
- **Security Challenges:** Emphasizes the need for lightweight security mechanisms that are suitable for low-power devices, which are common in IoT environments.

MQTT Implementations and Ecosystem



MQTT Implementations and Ecosystem

- **Brokers:** e.g. Mosquitto, HiveMQ, EMQX
- **Client Libraries:** MQTT client libraries like paho-mqtt in Python
- **MQTT for Sensor Networks (MQTT-SN):** a variant of MQTT optimized for wireless sensor networks.
- **Monitoring and Tools:** MQTT Explorer and MQTT.fx, which are useful for testing and monitoring MQTT communications.
- **Logging and Monitoring:** Crucial for maintaining a robust MQTT infrastructure.

MQTT Sensor Example - Components

Internet of Things (IoT) application example



Components Raspberry Pi Sense HAT

- 8x8 LED matrix display
- Accelerometer (ST Micro LSM9DS1)
- Gyroscope
- Magnetometer
- Air Pressure (260hPa to 1260hPa)
- Temperature (0-65°C)
- Humidity Sensor (0-100% relative humidity)
- Small Joystick

Python library «sense_hat» is used to access data

MQTT Sensor Example - Broker Commands

Internet of Things (IoT) application example

Command to start mosquitto broker

```
Eingabeaufforderung - mosq  
C:\Users\gell>mosquitto
```

Command to start the subscriber (topic = 'test/topic')

```
Eingabeaufforderung - mosq  
C:\Users\gell>mosquitto_sub -h localhost -t test/topic  
{temperature: 35, unit: degree_Celsius}
```

Command to start the publisher and send messages

```
Eingabeaufforderung  
C:\Users\gell>mosquitto_pub -d -t test/topic -m '{"temperature": 35, "unit": "degree_Celsius"}'  
Client null sending CONNECT  
Client null received CONNACK (0)  
Client null sending PUBLISH (d0, q0, r0, m1, 'test/topic', ... (39 bytes))  
Client null sending DISCONNECT  
C:\Users\gell>
```

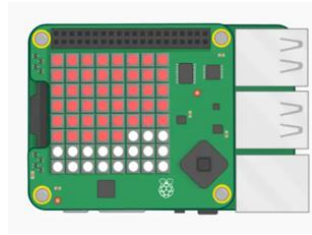
Example on Windows PC

MQTT Sensor Example - Publisher Setup

Internet of Things (IoT) application example

publisher.py

```
MQTT > ♦ mqtt_publisher.py > ...
1 import paho.mqtt.publish as publish
2 from sense_hat import SenseHat
3 import time
4
5 sense = SenseHat()
6
7 MQTT_SERVER = "127.0.0.1"
8 MQTT_TEMP = "sensor/temp"
9 MQTT_HUMI = "sensor/humi"
10 MQTT_PRES = "sensor/pres"
11
12 def read_temp():
13     t = sense.get_temperature()
14     t = round(t,2)
15     return t
16
17 def read_humidity():
18     h = sense.get_humidity()
19     h = round(h,2)
20     return h
21
22 def read_pressure():
23     p = sense.get_pressure()
24     p = round(p,2)
25     return p
26
27 temp = float(read_temp())
28 humi = float(read_humidity())
29 pres = float(read_pressure())
30
31 while True:
32     publish.single(MQTT_TEMP, temp, hostname=MQTT_SERVER)
33     time.sleep(0.5)
34     publish.single(MQTT_HUMI, humi, hostname=MQTT_SERVER)
35     time.sleep(0.5)
36     publish.single(MQTT_PRES, pres, hostname=MQTT_SERVER)
37     time.sleep(0.5)
38
```



Installations on Raspberry Pi

- Mosquitto (broker)
- paho-mqtt (Python library)
- Python

Installations on subscriber machine

- paho-mqtt (Python library)
- Python

MQTT Sensor Example - Subscriber Web Application

Internet of Things (IoT) application example

subscriber.py

```
MQTT > mqtt_subscriber.py > ...
1 import paho.mqtt.client as mqtt
2
3 # Suppress warnings
4 import warnings
5 warnings.filterwarnings("ignore")
6
7 # IP-Address of broker (Raspberry Pi)
8 MQTT_SERVER = "c1t-lab-w-1979"
9
10 # Topics
11 MQTT_TEMP = "sensor/temp"
12 MQTT_HUMI = "sensor/humi"
13 MQTT_PRES = "sensor/pres"
14
15 # The callback for when the client receives a CONNACK response from the server.
16
17 def on_connect(client, userdata, flags, rc):
18     print("Connected with result code " + str(rc))
19
20     # Subscribing in on_connect() means that if we lose the connection
21     # and reconnect then subscriptions will be renewed.
22     client.subscribe(MQTT_TEMP)
23     client.subscribe(MQTT_HUMI)
24     client.subscribe(MQTT_PRES)
25
26 # The callback for when a PUBLISH message is received from the server.
27
28 def on_message(client, userdata, msg):
29     msg = str(msg.topic) + " " + str(msg.payload)[2:-1]
30     print(msg)
31
32 client = mqtt.Client()
33 client.on_connect = on_connect
34 client.on_message = on_message
35
36 client.connect(MQTT_SERVER, 1883, 60)
37
38 # Loop
39 client.loop_forever()
40
```

MQTT

templates

- charts.html
- index.html

app.py

mqtt_publisher.py

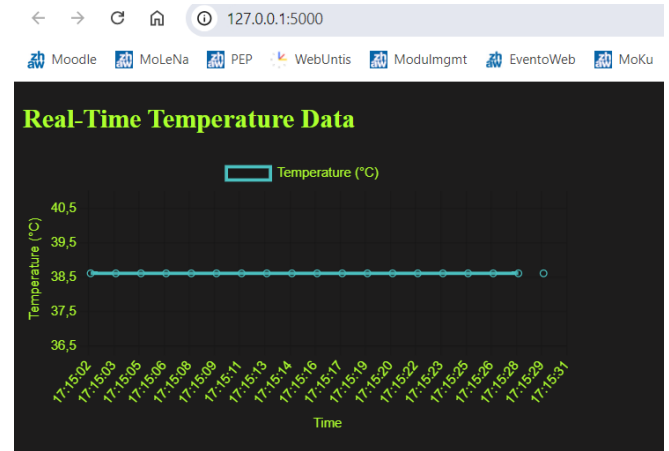
mqtt_subscriber.py

mosquitto_pub_sub.txt

requirements.txt



Web Application (Flask)



MQTT Example: Streaming Weather Data

Exercise: Event-Streaming with MQTT

GitHub repository: [Open the MQTT weather-data repository](#)

Steps:

- Fork the repository into your own GitHub account.
- Create a new GitHub Codespaces environment based on the repository.

Tasks (will be shown in class):

1. Run the example and try to understand the data and code.
2. Extend the code to implement “water temperature” and “wind speed” in the web app.

Apache Kafka Introduction

What is Apache Kafka?

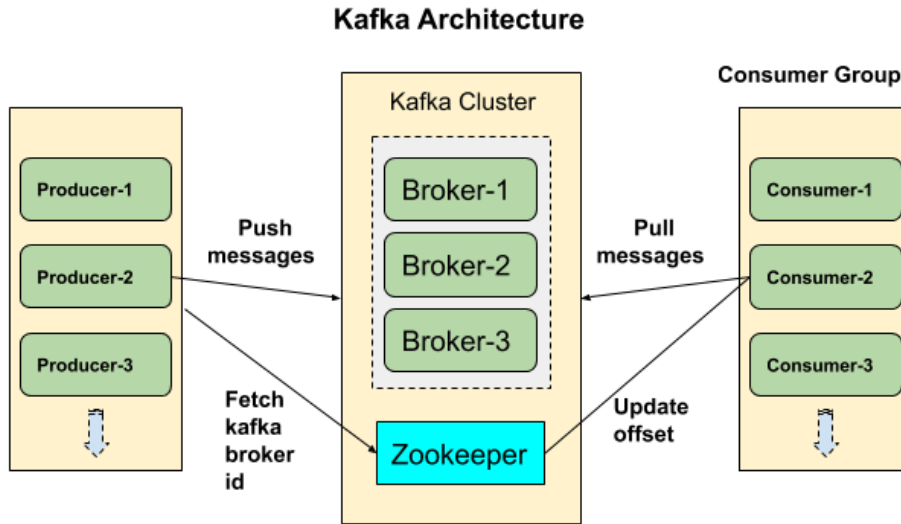
- Distributed Streaming Platform
- Publish and subscribe to streams of records
- Fault tolerant storage
- Replicates topic log partitions to multiple servers
- Process records as they occur
- Fast, efficient I/O operations, batching, compression
- Used to decouple data streams



Credit: Jean-Paul Azar (2017) Kafka Tutorial - Introduction to Apache Kafka (Part 1)

Apache Kafka: Architecture

Apache Kafka Main Components



Producer

- pushes data to the brokers

Consumer (= Subscriber)

- once subscribed to a topic, consumes the data

Kafka Broker

- producer does not directly interact with the consumer(s), they use Kafka broker as the agent to interact

Kafka Cluster

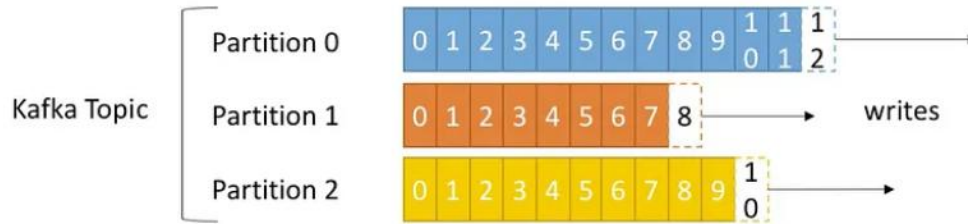
- group of computers each executing the same instance of Kafka broker

Zookeeper

- manages and coordinates the Kafka broker

Credit: <https://programmertoday.com/apache-kafka-architecture-and-components/>

Apache Kafka: Message Flows and Topics



In Kafka, the **message flow** begins with producers sending data to **topics**, which are logical channels for publishing data. These topics are further divided into **partitions** to enable parallel processing. Each message within a partition is assigned a unique **offset**, which helps consumers keep track of their position in the stream.

Image Credit: <https://catherine-shen.medium.com/scale-kafka-consumers-481be563bf3c>

Apache Kafka: Clustering and Scaling

Leader (red) and replicas (blue)

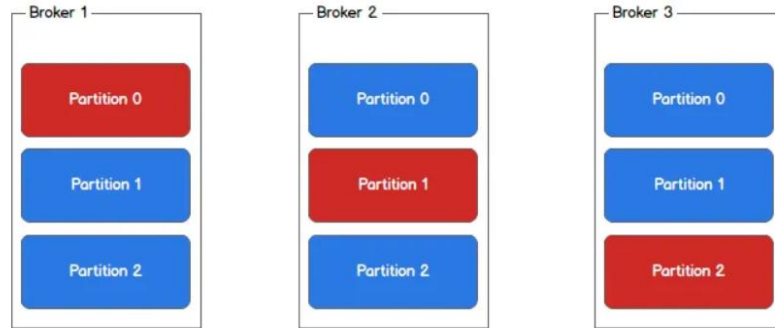


Image Credit: <https://catherine-shen.medium.com/scale-kafka-consumers-481be563bf3c>

Kafka scales horizontally* by adding more brokers to a cluster, with data distributed across partitions on these brokers to balance the load. It ensures fault tolerance through partition replication, where each partition has a leader and follower replicas. If a broker fails, a follower is promoted to leader, ensuring data availability. Kafka's design, including log-based storage and sequential I/O, allows it to handle a high volume of messages efficiently.

*increase capacity and performance by adding more brokers to the cluster, rather than upgrading existing machines.

Apache Kafka: Operations and Tools

Example end-to-end event streaming pipeline with Kafka

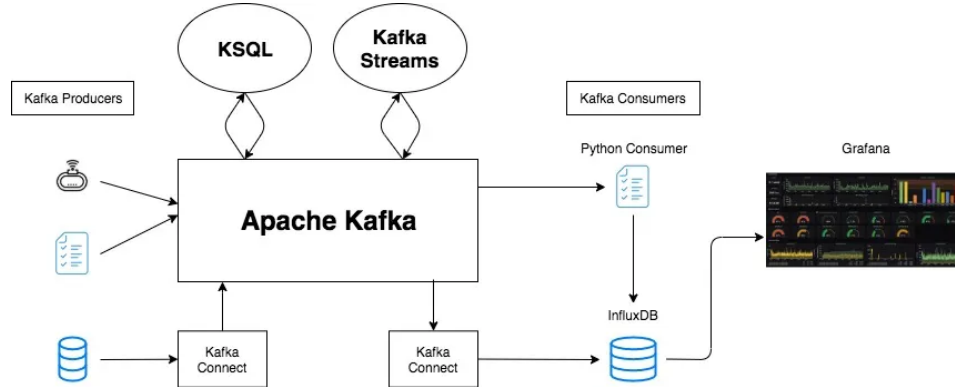


Image Credit: <https://medium.com/@ketulsheth2/streaming-data-pipeline-using-kafka-ksql-influxdb-and-grafana-8a934569fcb9>

Kafka uses a range of **operations and tools** to manage data streams efficiently. It offers high-level and low-level consumer APIs, with producers handling acknowledgments and ensuring message durability. The **Kafka Streams API** is used for building real-time stream processing applications, while **Kafka Connect** facilitates integration with external systems like databases. Monitoring and management are supported by tools like **Kafka Manager**, **Prometheus**, and **Grafana**. Additionally, Kafka faces operational challenges, such as handling rebalancing and tuning configurations to optimize performance.

Apache Kafka: Streaming Data Storage

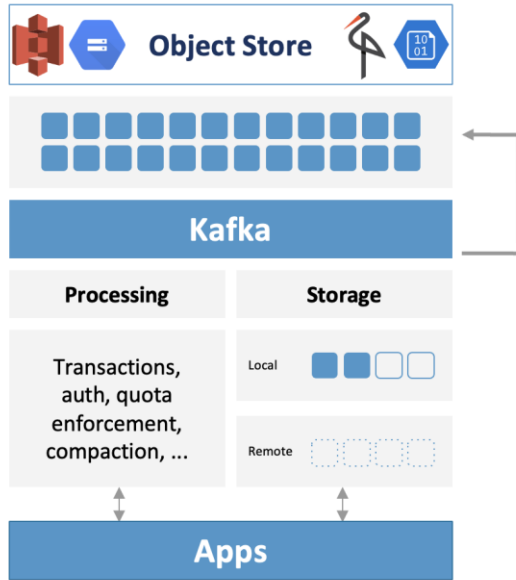
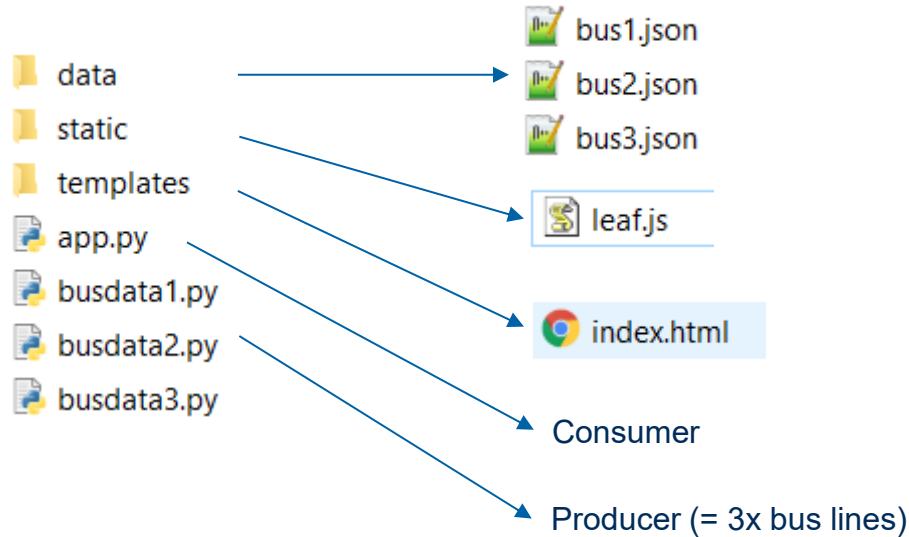


Image Credit: <https://www.kai-waehner.de/blog/2023/12/05/why-tiered-storage-for-apache-kafka-is-a-big-thing/>

Kafka provides a robust internal mechanism for storing data in topics with options like replication, log compaction, and retention policies. Additionally, Kafka integrates with various external storage systems to offload data for long-term storage, enabling a flexible and scalable data stream processing architecture. Most times, this is an object storage like Amazon S3, Azure Blob Storage, Google Cloud Storage, or MinIO for Kubernetes.

Kafka Bus-Routes Example - Architecture and Prerequisites

Prerequisites



Data about bus driving routes (3 routes)

JS-File (dynamic changes of HTML content)

HTML-File (map with buses in browser)

Kafka Bus-Routes Example - Docker Compose

docker-compose.yml

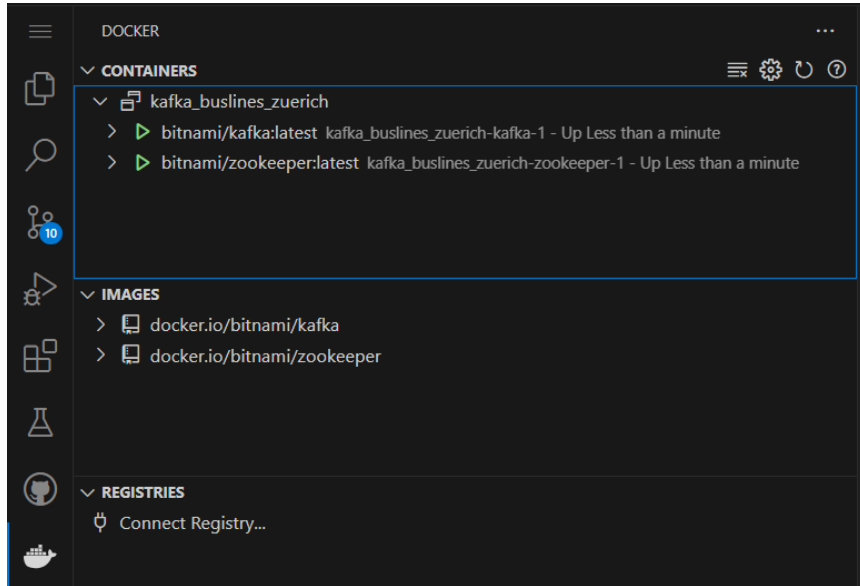
```
1 networks:
2   myNetwork:
3
4 services:
5
6   zookeeper:
7     image: 'bitnami/zookeeper:latest'
8     ports:
9       - '2181:2181'
10    environment:
11      - ALLOW_ANONYMOUS_LOGIN=yes
12    networks:
13      - myNetwork
14
15   kafka:
16     image: 'bitnami/kafka:latest'
17     user: root
18     ports:
19       - '9092:9092'
20     environment:
21       - KAFKA_BROKER_ID=1
22       - KAFKA_LISTENERS=PLAINTEXT://:9092
23       - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://127.0.0.1:9092
24       - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
25       - ALLOW_PLAINTEXT_LISTENER=yes
26       - KAFKA_CREATE_TOPICS=busdata001:1:1
27     volumes:
28       - ./Kafka:/bitnami/kafka
29     networks:
30       - myNetwork
31     depends_on:
32       - zookeeper
```

zookeeper

kafka

docker-compose.yml
with zookeeper and
kafka as separate
services

Kafka Bus-Routes Example - Running Containers



zookeeper and kafka
containers running on
GitHub Codespaces

```
@mario-gellrich-zhaw → /workspaces/kafka_buslines_zuerich (master) $ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
235dde3d70a0	bitnami/kafka:latest	"/opt/bitnami/script..."	About a minute ago	Up About a minute	0.0.0.0:9092->9092/tcp, :::9092->9092/tcp
d64313478828	bitnami/zookeeper:latest	"/opt/bitnami/script..."	About a minute ago	Up About a minute	2888/tcp, 3888/tcp, 0.0.0.0:2181->2181/tcp, :::2181->2181/tcp, 8080/tcp

Kafka Bus-Routes Example - Producer and Consumer Implementation

```
(adsenv) U:\Lektionen\ADS_FS2023\Dokumente_KK\Week_02\Kafka_Data_Streaming\Kafka_BusLines_zuerich>python busdata1.py
[\"busline\": \"00001\", \"key\": \"00001_75d6683-94d2-4e8f-8008-95461568d37\", \"timestamp\": \"2023-02-14 15:27:35.283335\", \"latitude\": 47.3671701707846185, \"longitude\": 8.544487953186935},
[\"busline\": \"00001\", \"key\": \"00001_6019a5df-762c-4794-9827-85321302399\", \"timestamp\": \"2023-02-14 15:27:35.471513\", \"latitude\": 47.3678095661158085, \"longitude\": 8.545947074890137},
[\"busline\": \"00001\", \"key\": \"00001_95a466c0-9072-4d28-a007-b75cf31bc900\", \"timestamp\": \"2023-02-14 15:27:36.382509\", \"latitude\": 47.36879780455503, \"longitude\": 8.547706604003906},
[\"busline\": \"00001\", \"key\": \"00001_40a67276-5016-4176-ab38-783178-62a76\", \"timestamp\": \"2023-02-14 15:27:36.800474\", \"latitude\": 47.3698194040000000, \"longitude\": 8.548647876373701},
[\"busline\": \"00001\", \"key\": \"(adsenv) U:\\Lektionen\\ADS_FS2023\\Dokumente_KK\\Week_02\\Kafka_Data_Streaming\\Kafka_BusLines_zuerich>python busdata2.py\", \"timestamp\": \"2023-02-14 15:34:44.293405\", \"latitude\": 47.36632717373784, \"longitude\": 8.540711402893066},
[\"busline\": \"00001\", \"key\": \"00002_47820977-0285-Ac1f-0701-bc93c7b0e972\", \"timestamp\": \"2023-02-14 15:34:44.810568\", \"latitude\": 47.36698089004721, \"longitude\": 8.543629646301277},
[\"busline\": \"00001\", \"key\": \"00002_459a5e04-21ff-4624-80c9-5481608f5013\", \"timestamp\": \"2023-02-14 15:34:45.321268\", \"latitude\": 47.36737357268174, \"longitude\": 8.544538868530273},
[\"busline\": \"00001\", \"key\": \"00002_23c9f046-47f1-408f-b02f-83c5c7b07a5e\", \"timestamp\": \"2023-02-14 15:34:45.833672\", \"latitude\": 47.37013408356699, \"longitude\": 8.54328832542362},
[\"busline\": \"00001\", \"key\": \"00002_c3058a43-47d9-417f-ab36-94e7f8c89902\", \"timestamp\": \"2023-02-14 15:34:46.343474\", \"latitude\": 47.373007728231776, \"longitude\": 8.54285717010498},
[\"busline\": \"00001\", \"key\": \"00002_860fb38c-3249-415d-8507-11e07c0004\", \"timestamp\": \"2023-02-14 15:34:46.856887\", \"latitude\": 47.37378965616847, \"longitude\": 8.54281425476042},
[\"busline\": \"00001\", \"key\": \"00002_9d40b601-5161-4843-8918-320d05a1e401\", \"timestamp\": \"2023-02-14 15:34:47.364311\", \"latitude\": 47.37519184186028, \"longitude\": 8.543286323547362},
[\"busline\": \"00001\", \"key\": \"(adsenv) U:\\Lektionen\\ADS_FS2023\\Dokumente_KK\\Week_02\\Kafka_Data_Streaming\\Kafka_BusLines_zuerich>python busdata3.py\", \"timestamp\": \"2023-02-14 15:36:09.450834\", \"latitude\": 47.356792368516766, \"longitude\": 8.551568984985352},
[\"busline\": \"00001\", \"key\": \"00003_ba8c678b-3339-4d0f-8419-0091b1e0b33e\", \"timestamp\": \"2023-02-14 15:36:09.370573\", \"latitude\": 47.35824594046403, \"longitude\": 8.556718826293945},
[\"busline\": \"00001\", \"key\": \"00003_262b58da-2026-4306-9089-17e349d3a39b\", \"timestamp\": \"2023-02-14 15:36:09.880366\", \"latitude\": 47.35940879179006, \"longitude\": 8.55783462524414},
[\"busline\": \"00001\", \"key\": \"00003_33f64b88-4808-47a8-a452-352fcb4b1ecc\", \"timestamp\": \"2023-02-14 15:36:07.392931\", \"latitude\": 47.36004834414535, \"longitude\": 8.560665703796387},
[\"busline\": \"00001\", \"key\": \"00003_37a05408-4906-4815-8714-6ef4230c3dbf\", \"timestamp\": \"2023-02-14 15:36:07.986675\", \"latitude\": 47.36138540492764, \"longitude\": 8.561439514168156},
[\"busline\": \"00001\", \"key\": \"00003_dfc8640e-49be-471a-9a20-29acab336d3b\", \"timestamp\": \"2023-02-14 15:36:08.420908\", \"latitude\": 47.362025993328025, \"longitude\": 8.56058120727539},
[\"busline\": \"00001\", \"key\": \"00003_934c4245-214c-462a-87d5-716dc2a051bf\", \"timestamp\": \"2023-02-14 15:36:08.932219\", \"latitude\": 47.36336262600012, \"longitude\": 8.564443588256836},
[\"busline\": \"00001\", \"key\": \"00003_e0dc4cf2f-72f5-4346-b54d-4d5f0e2c9ab\", \"timestamp\": \"2023-02-14 15:36:09.442237\", \"latitude\": 47.36423431357495, \"longitude\": 8.56607437133789},
[\"busline\": \"00001\", \"key\": \"00003_7929681-3339-4d0f-8419-0091b1e0b33e\", \"timestamp\": \"2023-02-14 15:36:09.951127\", \"latitude\": 47.364948007821197, \"longitude\": 8.5663318630032},
[\"busline\": \"00001\", \"key\": \"00003_63d3070d-a441-4001-b986-f2cc4d5e3833\", \"timestamp\": \"2023-02-14 15:36:10.465781\", \"latitude\": 47.367547970487536, \"longitude\": 8.564615240613780},
[\"busline\": \"00001\", \"key\": \"00003_0882d455-123f-4494-45a0-30efb577865\", \"timestamp\": \"2023-02-14 15:36:10.974527\", \"latitude\": 47.36975695949515, \"longitude\": 8.563928604125977},
[\"busline\": \"00001\", \"key\": \"00003_31073ce3-e08d-4443-b762-dac3626844d1\", \"timestamp\": \"2023-02-14 15:36:11.486703\", \"latitude\": 47.3704545156932, \"longitude\": 8.56358528137289},
[\"busline\": \"00001\", \"key\": \"00003_8d0e7231-7d6c-4c2b-b3ec-3920ee8604f8\", \"timestamp\": \"2023-02-14 15:36:11.997569\", \"latitude\": 47.37170472702227, \"longitude\": 8.56201508517578},
[\"busline\": \"00001\", \"key\": \"00003_c770810e-e27c-4d13-ae1c-2a0a1b3d84c0\", \"timestamp\": \"2023-02-14 15:36:12.508125\", \"latitude\": 47.37266726607714, \"longitude\": 8.563041950038164},
[\"busline\": \"00001\", \"key\": \"00003_f45da3ce-6efa-4343-bc35-d4f6a2a10e07\", \"timestamp\": \"2023-02-14 15:36:13.017632\", \"latitude\": 47.37353527873642, \"longitude\": 8.561525344848633},
[\"busline\": \"00001\", \"key\": \"00003_1d2bcff7-e9f3-4bac-b001-dc71a01d0311\", \"timestamp\": \"2023-02-14 15:36:13.530408\", \"latitude\": 47.37550646021242, \"longitude\": 8.560152053833008},
[\"busline\": \"00001\", \"key\": \"00003_6c6e8a73-43d5-4801-8401-dbe4fd45f763\", \"timestamp\": \"2023-02-14 15:36:14.042659\", \"latitude\": 47.376092756617446, \"longitude\": 8.5594654081325195},
[\"busline\": \"00001\", \"key\": \"00003_73a1a248-4079-4c92-bc5d-c302224f4c3a\", \"timestamp\": \"2023-02-14 15:36:14.557318\", \"latitude\": 47.376515123239966, \"longitude\": 8.557486547180724},
[\"busline\": \"00001\", \"key\": \"00003_a7a0423a-9d84-4408-8444-b7a8f178353a\", \"timestamp\": \"2023-02-14 15:36:15.068252\", \"latitude\": 47.37684835134525, \"longitude\": 8.554487228393555},
[\"busline\": \"00001\", \"key\": \"00003_27c768b9-8379-4a8f-9a20-027f4db66d3\", \"timestamp\": \"2023-02-14 15:36:15.581434\", \"latitude\": 47.37702271782176, \"longitude\": 8.552942276080077},
[\"busline\": \"00001\", \"key\": \"00003_3d07bf60-44af-44e5-b661-d0e4ffdb037e\", \"timestamp\": \"2023-02-14 15:36:16.097530\", \"latitude\": 47.37804006375369, \"longitude\": 8.553285598754883},
[\"busline\": \"00001\", \"key\": \"00003_86a189ec-2633-4a07-8e0b-efbfc0e0ebd0\", \"timestamp\": \"2023-02-14 15:36:16.613302\", \"latitude\": 47.3781493331074, \"longitude\": 8.55310230779250},
[\"busline\": \"00001\", \"key\": \"00003_c692a09e-9169-45ea-b836-dff801397aab\", \"timestamp\": \"2023-02-14 15:36:17.126813\", \"latitude\": 47.374755988651956, \"longitude\": 8.550195693697272}
```

 busdata1.py

 busdata2.py

 busdata3.py

```
(adsenv) U:\\Lektionen\\ADS_FS2023\\Dokumente_KK\\Week_02\\Kafka_Data_Streaming\\Kafka_BusLines_zuerich>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 813-142-908
```

 app.py

Kafka Bus-Routes Example - Browser Visualization

```
51 #-----  
52 # Browser-Map  
53 #-----  
54  
55 http://127.0.0.1:5001  
56 http://127.0.0.1:5001/topic/busdata\_001
```

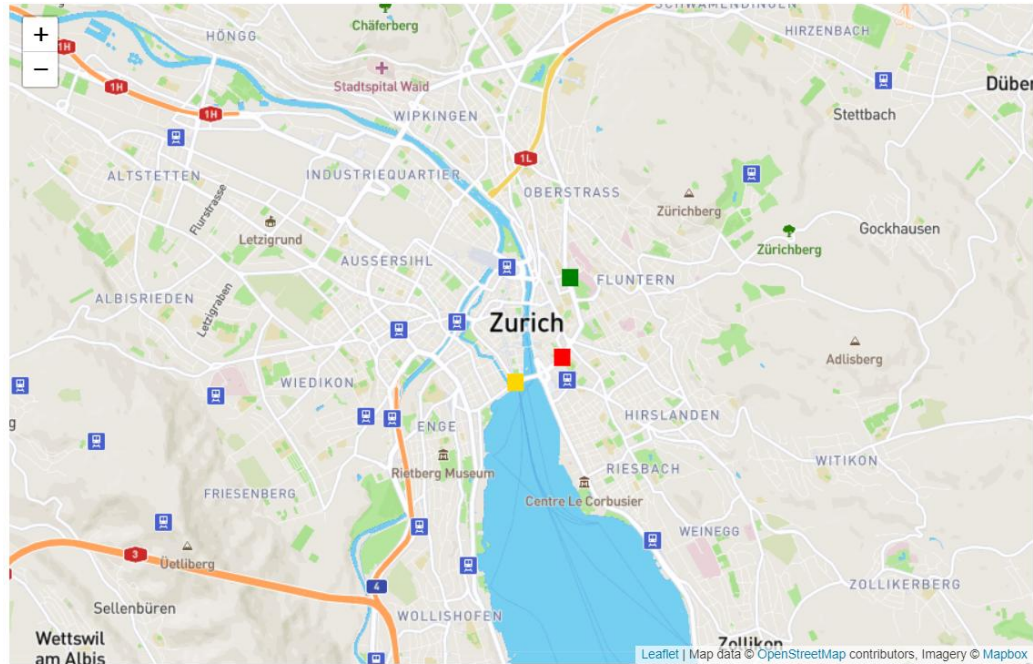


< > × 📏 🌐 127.0.0.1:5001/topic/busdata_001

data:{"busline": "00001", "key": "00001_55cafe12-8d86-4ca3-81ef-2726b33d5cf", "timestamp": "2021-02-23 10:13:00.795886", "latitude": 47.367170107846185, "longitude": 8.544487953186035}
data:{"busline": "00001", "key": "00001_7413d12c-48e2-49ac-a3f4-66571a0c9a78", "timestamp": "2021-02-23 10:13:01.346313", "latitude": 47.367809566115085, "longitude": 8.545947074890137}
data:{"busline": "00001", "key": "00001_ac54e38d-7815-4a6b-8b0d-7edac47993db", "timestamp": "2021-02-23 10:13:01.849241", "latitude": 47.36879780455503, "longitude": 8.547706604003906}
data:{"busline": "00001", "key": "00001_38e30157-6fb9-47af-80c4-d24874b35730", "timestamp": "2021-02-23 10:13:02.356983", "latitude": 47.36981508949069, "longitude": 8.54860782623291}
data:{"busline": "00001", "key": "00001_f12d95ed-33c5-4003-9b58-ced8db165d08", "timestamp": "2021-02-23 10:13:02.859408", "latitude": 47.37065796785916, "longitude": 8.54959487915039}
data:{"busline": "00001", "key": "00001_54c8d358-c05a-4809-9d28-58714137f65c", "timestamp": "2021-02-23 10:13:03.362241", "latitude": 47.37184960049907, "longitude": 8.550281524658203}
data:{"busline": "00001", "key": "00001_e413df29-640b-49eb-9d1e-3f9b96d93a77", "timestamp": "2021-02-23 10:13:03.864658", "latitude": 47.372401811373194, "longitude": 8.55062484741211}
data:{"busline": "00001", "key": "00001_7293608a-e18e-4550-9109-5c3e9097ea5d", "timestamp": "2021-02-23 10:13:04.367006", "latitude": 47.374581534679564, "longitude": 8.549938201904297}
data:{"busline": "00001", "key": "00001_319020d1-83ae-4ed7-8522-d710d037a7a4", "timestamp": "2021-02-23 10:13:04.869346", "latitude": 47.37626712559255, "longitude": 8.548994064331055}
data:{"busline": "00001", "key": "00001_bf093d34-4eff-4f77-83a0-30327696c7be", "timestamp": "2021-02-23 10:13:05.372570", "latitude": 47.37809796501885, "longitude": 8.548221588134764}

Go to a web browser and visualize the data and map

Kafka Bus-Routes Example - Real-Time Map



Map with real-time data

Kafka Bus-Routes Example - Exercise

Exercise: Event-Streaming with Kafka

GitHub repository: [Open the Kafka bus-routes repository](#)

Steps:

- Fork the repository into your own GitHub account.
- Create a new GitHub Codespaces environment based on the repository.

Tasks (will be shown in class):

1. Run the example and try to understand the data and code.
2. Integrate one additional bus line into the web app.

Key Takeaways - MQTT and Apache Kafka

You can ...

- explain the basics of MQTT, including its architecture, protocol details, and security measures.
- identify practical applications of MQTT in IoT and other domains.
- use the paho-mqtt Python library to subscribe to real-time IoT sensor data.
- explain the fundamentals of Apache Kafka, including its architecture and core concepts.
- explain message flow, topics, and how data is managed in Kafka.
- explain how Kafka handles clustering and scaling for high availability.
- explain key Kafka operations and tools for managing data streams.
- implement Kafka in real-world examples (such as live bus routes in Zürich).

Literature - MQTT and Apache Kafka

- Wingerath, W., Gessert, F., Friedrich, S., & Ritter, N. (2016). Real-time stream processing for Big Data. *it-Information Technology*, 58(4), 186-194.
- Chakravarthy, S., & Jiang, Q. (2009). *Stream data processing: A quality of service perspective. Modeling, scheduling, load shedding, and complex event processing.* Springer.
- Moreira-Matias, L., et al. (2013). Predicting taxi-passenger demand using streaming data. *IEEE Transactions on Intelligent Transportation Systems*, 14(3), 1393–1402.
- Apache Kafka (n.d.). Introduction: Everything you need to know about Kafka in 10 minutes. <https://kafka.apache.org/intro>
- Desbiens, F. (2023). *Building enterprise IoT solutions with Eclipse IoT technologies.* Apress.